

OncoGuard-AI

Breast Cancer Diagnostic Classification & Model Comparison

Technical Project Report

Prepared by Anum Naveed

AI / ML Engineer — Computer Vision & Applied Machine Learning

August 2026

Abstract

Early and accurate diagnosis is one of the strongest levers for improving breast cancer outcomes. This project builds a machine-learning-based classification system on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset that labels a case as BENIGN or MALIGNANT from 30 numerical diagnostic features. Rather than committing to a single algorithm, five classical classifiers — Logistic Regression, SVM, Random Forest, KNN, and Decision Tree — are trained on an identical 80/20 stratified split and compared across seven evaluation metrics. Random Forest was automatically selected as the best model, reaching 97.4% accuracy and a 0.998 ROC-AUC on the held-out test set. The best model, its comparison table, confusion matrix, and ROC curves are saved for reuse, and the pipeline supports probability-based prediction for a new patient's feature values.

Introduction

Breast cancer detection involves determining whether a breast lesion is benign or malignant. Machine learning can analyze numerical diagnostic features and learn the patterns that separate these two classes. This project implements a supervised learning approach: the input is a set of numerical features from the WDBC dataset, and the target is the diagnosis, encoded as 0 = BENIGN and 1 = MALIGNANT. Rather than relying on a single model, the project deliberately compares multiple classification algorithms side by side and lets the data determine the best performer.

Problem Statement

Given a set of diagnostic breast-cancer features, determine whether the corresponding case belongs to the benign or malignant class — and provide a quantitative, multi-metric evaluation so that different machine-learning algorithms can be fairly compared on the same data.

Objective

To develop and compare machine-learning classification models for distinguishing benign and malignant breast cancer cases using diagnostic features from the Wisconsin Diagnostic Breast Cancer Dataset.

- Preprocess and validate the dataset.
- Train multiple ML algorithms on an identical split.
- Compare their performance using clinically relevant metrics.
- Select and save the best-performing model automatically.
- Support prediction for a new patient's feature values.

Scope

- Load the Wisconsin Diagnostic Breast Cancer Dataset and check it for missing values.

- Examine class distribution and prepare features / target labels.
- Split data into stratified training and testing sets, scaling features where required.
- Train five machine-learning algorithms and compare their performance.
- Calculate multiple classification metrics, a confusion matrix, and ROC curves.
- Select the best model by ROC-AUC and save it for reuse.
- Perform prediction for a new patient's feature values.

The current implementation works on structured diagnostic features — it is not an image-based tumor-detection system and does not perform CNN-based image classification.

Dataset

The project uses the Wisconsin Diagnostic Breast Cancer Dataset, supplied as `breast_cancer_dataset.csv`. It contains 569 patient records, each described by 30 real-valued diagnostic features computed from digitized images of breast mass fine-needle aspirates. The diagnosis column is separated from the remaining feature columns and used as the target.

Property	Value
Total samples	569
Input features	30 numerical diagnostic features
Target classes	BENIGN (0), MALIGNANT (1)
Train / test split	80% / 20%, stratified (test set = 114 samples)
Random state	42

- The program checks dataset shape, the first five records, dtypes, and missing values before use.
- No missing values were found in any column.
- Stratification (`stratify = y`) preserves the class balance between the training and testing subsets.
- The new-patient prediction function expects exactly 30 feature values, in the same order as the CSV feature columns.

Methodology

- Dataset loading — read `breast_cancer_dataset.csv` with Pandas.
- Dataset inspection — shape, head, dtypes, missing values, class distribution.
- Feature / target preparation — separate the diagnosis column from the input features.
- Label encoding — BENIGN → 0, MALIGNANT → 1.
- Dataset splitting — 80/20 stratified train/test split.
- Model training — five classifiers trained independently on the same training data.
- Prediction — each trained model predicts on the held-out test set.
- Evaluation — accuracy, precision, recall, sensitivity, specificity, F1-score, and ROC-AUC.
- Model comparison — results ranked by ROC-AUC, descending.

- Best-model selection — the model with the highest ROC-AUC is chosen automatically.
- Deployment-ready prediction — the selected model accepts 30 feature values for a new case and returns benign/malignant probabilities.

Preprocessing

- Missing-value check across every column via `data.isnull().sum()` — none found.
- Diagnosis encoded from text to numeric labels: BENIGN → 0, MALIGNANT → 1.
- 80/20 stratified split, using `test_size = 0.20`, `random_state = 42`, `stratify = y`.

There is no image augmentation in this project, since it works with structured numerical features rather than image pixels.

Feature Scaling

StandardScaler is applied to Logistic Regression, SVM, and KNN, combined with each model inside a Scikit-learn Pipeline so that scaling and classification are fit together without leaking test-set information. Random Forest and Decision Tree are tree-based and are used directly, without standardization.

Models & Hyperparameters

Five classifiers are trained independently on the identical training split and evaluated on the identical test split, which keeps the comparison fair:

Model	Key Hyperparameters
Logistic Regression	max_iter = 5000, random_state = 42
SVM	kernel = "rbf", probability = True, random_state = 42
Random Forest	n_estimators = 300, random_state = 42, class_weight = "balanced"
KNN	n_neighbors = 5
Decision Tree	random_state = 42, class_weight = "balanced"

Tools & Libraries

- Python, with Scikit-learn for preprocessing, model training, and evaluation
- NumPy, Pandas — numerical computation and dataset / results handling
- Matplotlib — confusion matrix, ROC curves, and feature-importance plots
- Joblib — saving the trained model as `best_breast_cancer_model.pkl`
- Scikit-learn model classes — `LogisticRegression`, `SVC`, `RandomForestClassifier`, `KNeighborsClassifier`, `DecisionTreeClassifier`
- Scikit-learn metrics — `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `confusion_matrix`, `classification_report`, `roc_auc_score`, `roc_curve`
- Development environment — Visual Studio Code with the Python debugger

Pipeline Overview

- Load the CSV dataset and inspect it (shape, head, dtypes, missing values, class distribution).
- Separate features from the diagnosis target and encode BENIGN/MALIGNANT as 0/1.
- Split 80% training / 20% testing, stratified by class.
- Standardize features for Logistic Regression, SVM, and KNN inside a Pipeline.
- Train all five classifiers on the training set.
- Predict on the test set and compute accuracy, precision, recall, sensitivity, specificity, F1, and ROC-AUC for each model.
- Rank all models by ROC-AUC and select the best one.
- Generate the confusion matrix and ROC curves for the best model, plus feature importance if it is Random Forest.
- Save the best model as `best_breast_cancer_model.pkl` and the comparison table as `model_comparison_results.csv`.
- Use the saved model to predict benign/malignant probabilities for a new patient's 30 feature values.

Results

All five models were trained on the same 455-sample training set and evaluated on the same 114-sample held-out test set. The table below (sorted by ROC-AUC, descending, matching `model_comparison_results.csv`) reports all seven metrics; the best model is highlighted.

Model	Accuracy	Precision	Recall	Sensitivity	Specificity	F1	ROC-AUC
Random Forest	0.9737	1.0000	0.9286	0.9286	1.0000	0.9630	0.9980
Logistic Regression	0.9649	0.9750	0.9286	0.9286	0.9861	0.9512	0.9960
SVM	0.9737	1.0000	0.9286	0.9286	1.0000	0.9630	0.9947
KNN	0.9561	0.9744	0.9048	0.9048	0.9861	0.9383	0.9823
Decision Tree	0.9035	0.8974	0.8333	0.8333	0.9444	0.8642	0.8889

Random Forest achieved the highest ROC-AUC (0.998) and was selected automatically as the best model. It ties SVM on point-prediction metrics (accuracy, precision, recall, specificity, F1) but ranks predictions more reliably, which is what the AUC captures. Precision is reported for the malignant (positive) class.

Per-Class Report — Best Model (Random Forest)

Class	Precision	Recall	F1-Score	Support
Benign	0.96	1.00	0.98	72
Malignant	1.00	0.93	0.96	42
Accuracy			0.97	114
Macro avg	0.98	0.96	0.97	114
Weighted avg	0.97	0.97	0.97	114

Confusion Matrix — Random Forest

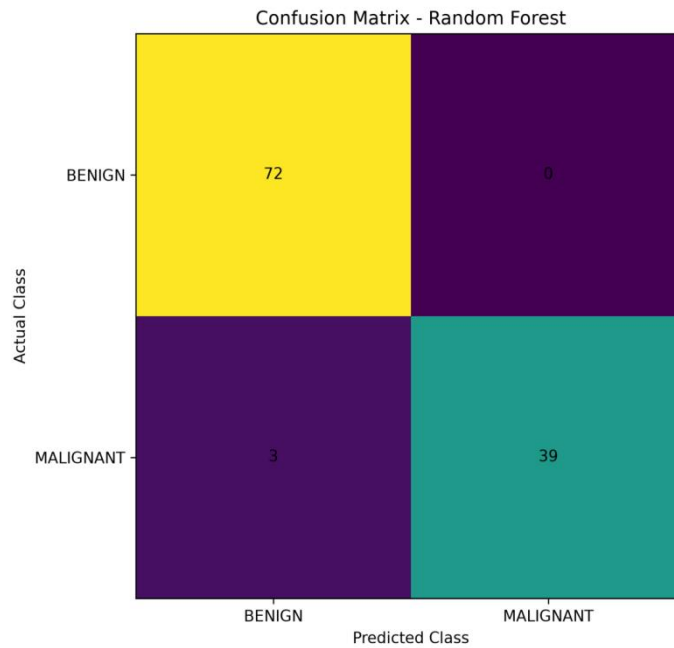


Figure 1. Test-set confusion matrix for the selected best model (114 test cases). Only 3 malignant cases were missed; no benign case was misclassified as malignant.

ROC Curve Comparison

This project uses classical (non-epoch-based) machine-learning algorithms, so conventional CNN-style training curves (accuracy/loss vs. epoch) do not apply. The ROC curve comparison is the equivalent visualization for this pipeline.

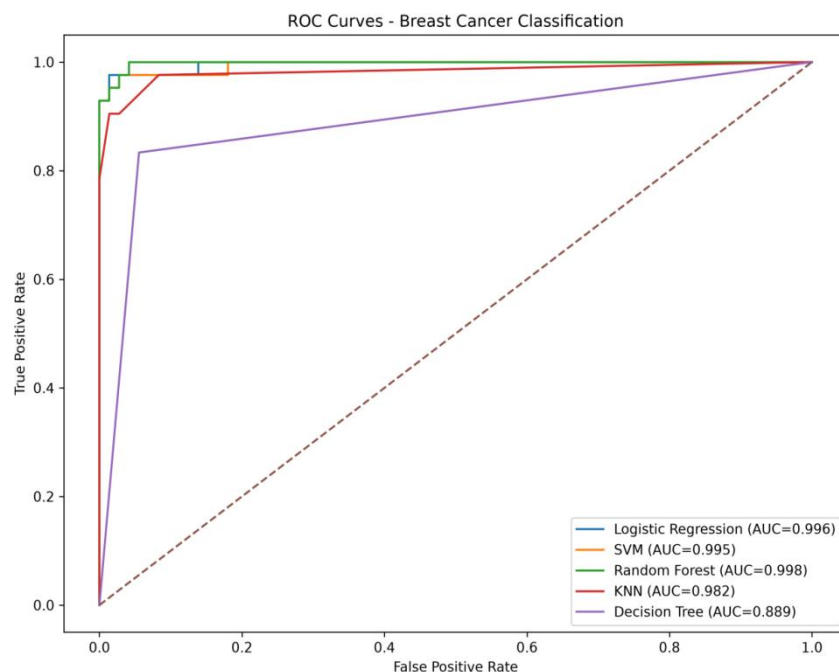


Figure 2. ROC curves for all five models. Random Forest (AUC = 0.998) and Logistic Regression (AUC = 0.996) lead; Decision Tree (AUC = 0.889) trails the rest.

Feature Importance

Because the best model is Random Forest, the pipeline also reports the top 10 most influential features.

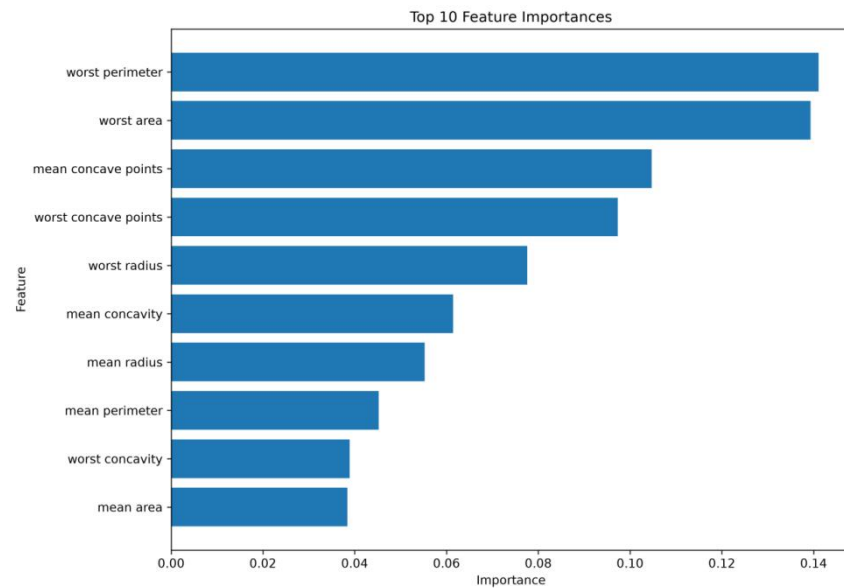


Figure 3. Top 10 feature importances. Worst perimeter and worst area dominate, followed by the concave-point measures — consistent with malignant tumors tending to be larger and less regular in shape.

Qualitative Results

Since the input is tabular diagnostic data rather than images, qualitative interpretation is based on the confusion matrix, ROC curves, and feature importance above, together with individual predictions for new patients. The pipeline also outputs benign/malignant probabilities for a new patient's 30 feature values, rather than a bare class label.

Discussion

- All five models perform well on this dataset, but they are not equivalent — Decision Tree lags noticeably (ROC-AUC 0.889 vs. ≥ 0.98 for the rest), consistent with a single tree overfitting more easily than an ensemble or a margin-based method.
- Random Forest and SVM are tied on point metrics (accuracy, precision, recall, F1), but Random Forest's higher ROC-AUC indicates better-calibrated confidence ranking across thresholds, which matters for triage-style use.
- Recall (sensitivity) for the malignant class is 0.93 for the best model — 3 of 42 malignant cases in the test set were missed. Precision is a perfect 1.00, meaning no benign case was flagged as malignant. In a clinical context the 3 missed cases are the more consequential error, since a false negative delays treatment.
- The best model is chosen automatically by ROC-AUC rather than by manually favoring one algorithm, keeping the selection process objective and reproducible.

Model Comparison

Model	Main Characteristic
Logistic Regression	Linear classification model
SVM	Nonlinear classification using an RBF kernel
Random Forest	Ensemble of 300 decision trees; captures nonlinear feature relationships
KNN	Classifies based on the 5 nearest neighboring samples
Decision Tree	Single rule-based tree; fastest but least robust here

Related Work

Logistic Regression, SVM, Decision Trees, Random Forests, and K-Nearest Neighbors are commonly used for binary classification on structured medical data. This project follows a comparative approach — training several classical algorithms on the same dataset and evaluating them with multiple performance measures — rather than committing to a single method or asserting improvement over specific prior published results.

Deployment — OncoGuard-AI

The classification pipeline was packaged into OncoGuard-AI, a Streamlit web application with a home page for viewing predictions and results, turning the saved model into an interactive, shareable tool rather than a script that only runs from the command line.

Limitations

- Uses tabular diagnostic features, not breast MRI, mammography, or ultrasound images.
- Evaluated with a single 80/20 train/test split rather than cross-validation.

- No automated hyperparameter search or systematic optimization is implemented.
- No external clinical validation, and no confidence intervals are reported for the metrics.
- Real-world clinical data streams may be less balanced than this research dataset.
- A machine-learning prediction should not be interpreted as a standalone medical diagnosis.

Important Disclaimer

- This system is a decision-support research prototype, not a diagnostic device.
- It does not replace a radiologist, oncologist, or pathologist's assessment.
- Trained and evaluated on a single public research dataset only, with no clinical trial validation or regulatory clearance.
- All predictions — for any patient — require qualified clinical review before acting on them.

Conclusion

This project implements a complete machine-learning pipeline for binary breast cancer classification. Five different algorithms were trained on the same 80/20 stratified split and evaluated on accuracy, precision, recall, sensitivity, specificity, F1-score, and ROC-AUC. Random Forest was automatically selected as the best model, reaching 97.4% accuracy and a 0.998 ROC-AUC, and was saved as a reusable .pkl file alongside a confusion matrix, ROC curve comparison, and feature-importance plot. Packaged as the OncoGuard-AI web app, the project provides a practical, comparative machine-learning framework for benign-versus-malignant classification — while actual clinical use would require further validation and expert medical oversight.

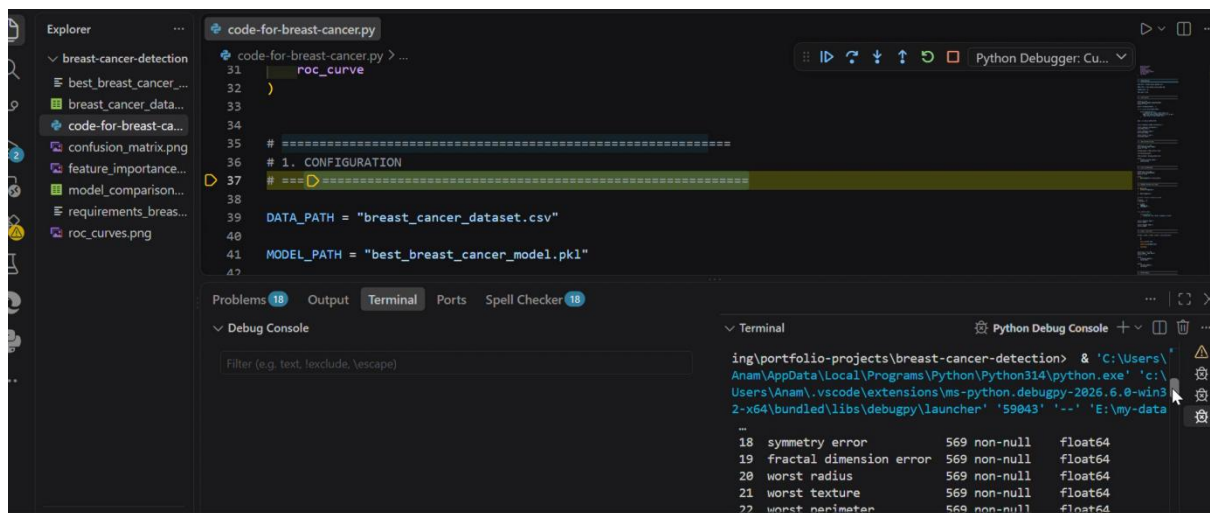
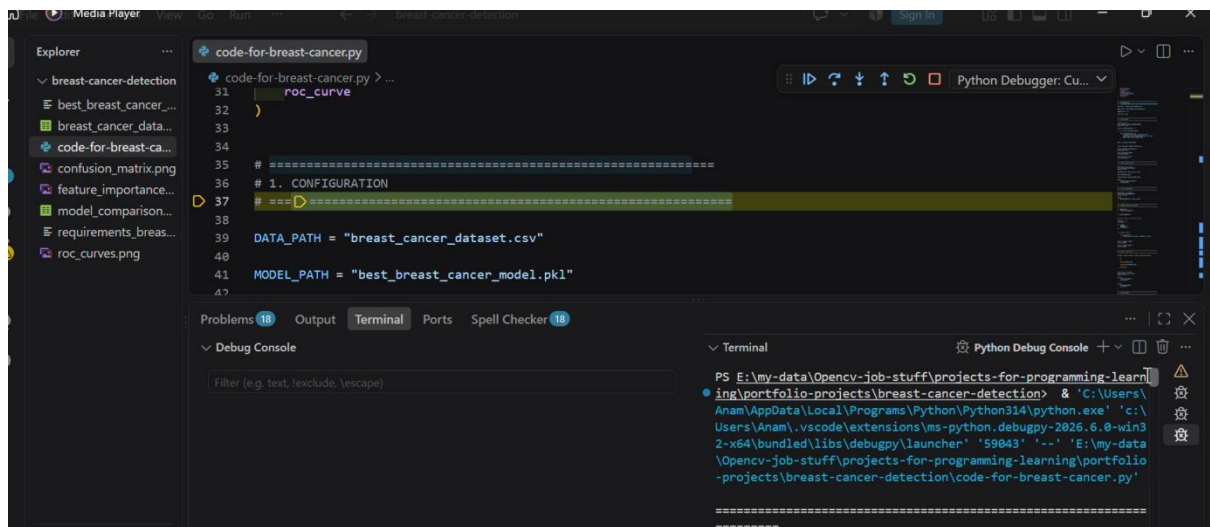
Key Points

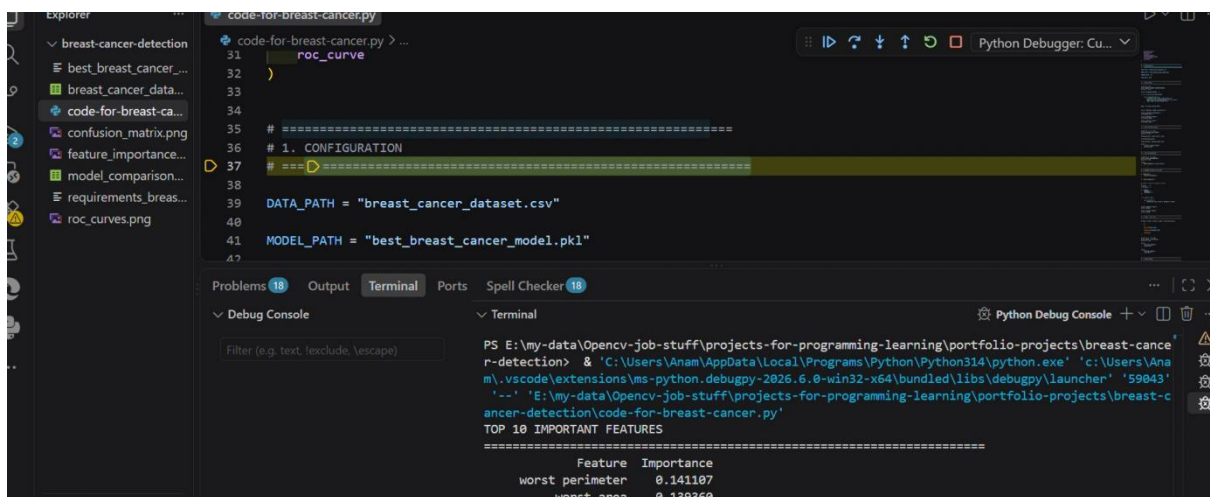
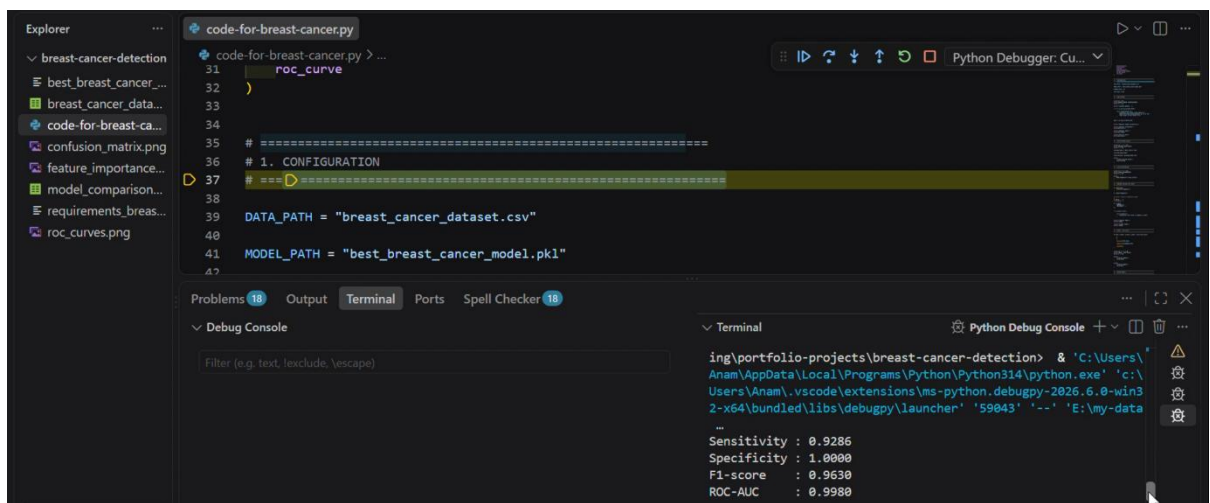
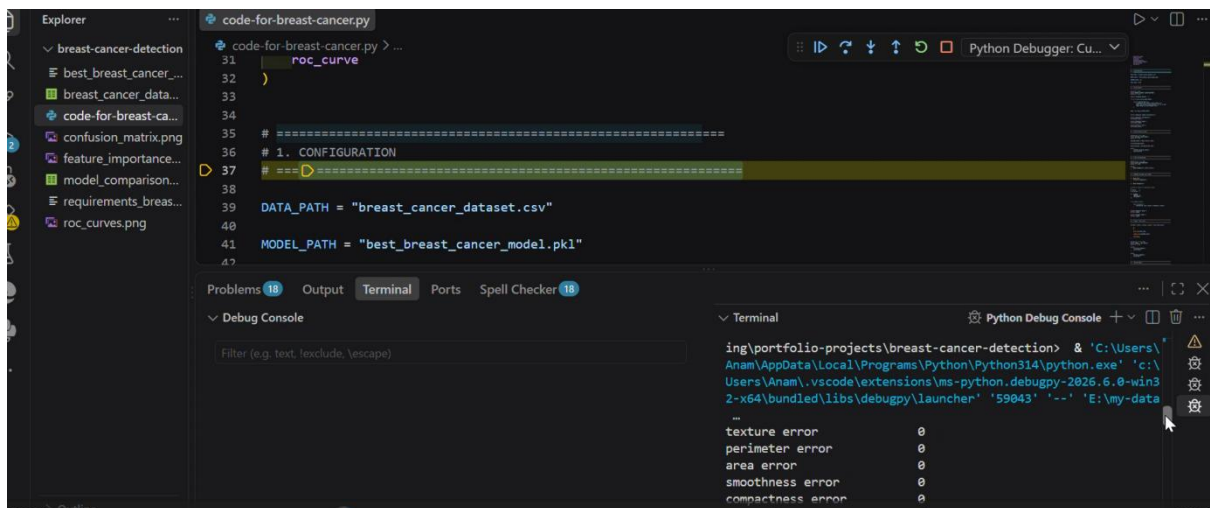
- Project: Breast Cancer Detection (OncoGuard-AI)
- Dataset: Wisconsin Diagnostic Breast Cancer Dataset — 569 samples, 30 features
- Problem type: Binary classification — BENIGN vs. MALIGNANT
- Train/test split: 80/20, stratified, random_state = 42
- Models compared: 5 (Logistic Regression, SVM, Random Forest, KNN, Decision Tree)
- Best model: Random Forest — Accuracy 0.9737, ROC-AUC 0.9980
- Feature scaling: StandardScaler for Logistic Regression, SVM, and KNN
- Model-selection criterion: highest ROC-AUC
- Visualization: confusion matrix, ROC curves, feature importance
- Model format: .pkl (Joblib) · Framework: Scikit-learn · Execution: CPU-based
- Deployment: Streamlit web app (OncoGuard-AI)

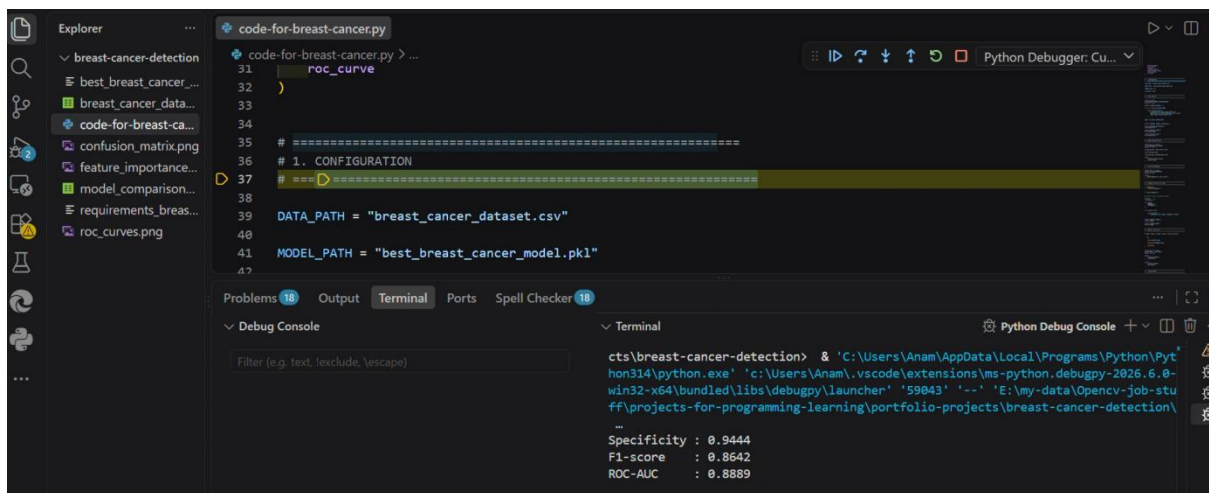
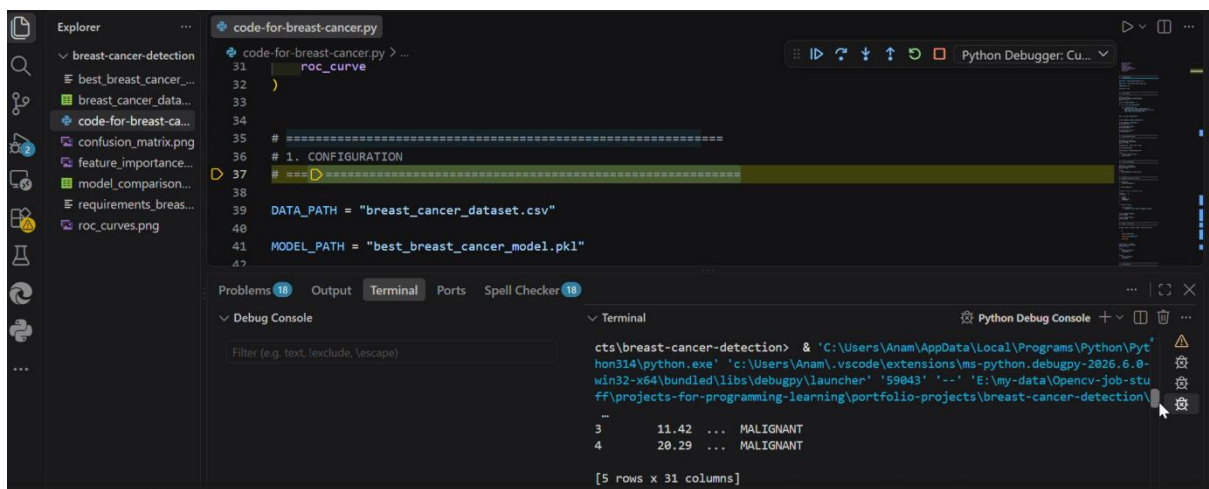
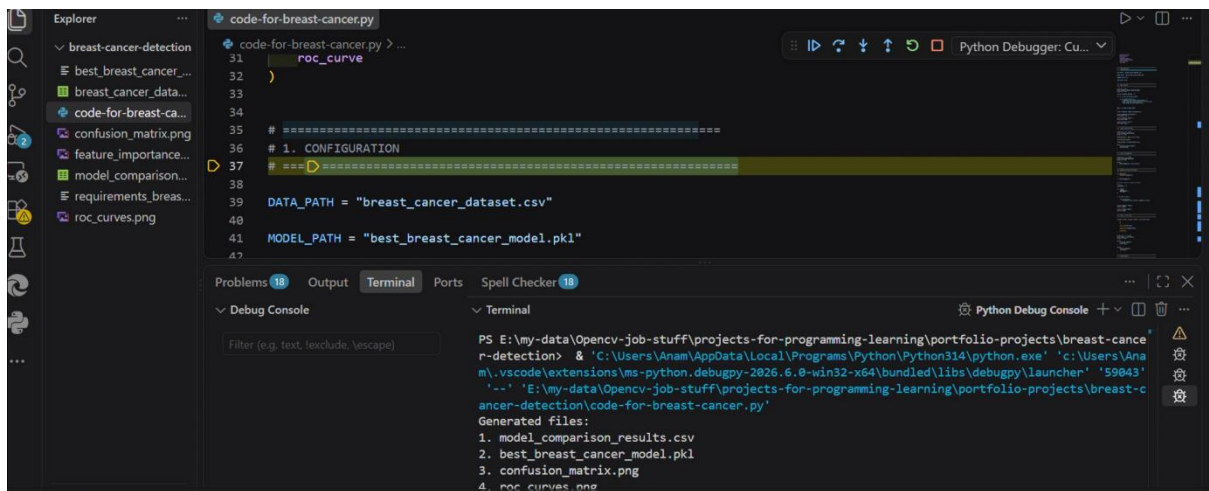
Outputs Generated

File	Contents
model_comparison_results.csv	Performance of all five models across seven metrics
best_breast_cancer_model.pkl	The saved best-performing model (Random Forest)
confusion_matrix.png	Confusion matrix for the best model, saved at 300 DPI
roc_curves.png	ROC curves for all five models
feature_importance.png	Top 10 feature importances (generated when Random Forest is best)

Process Images on visual studio code:





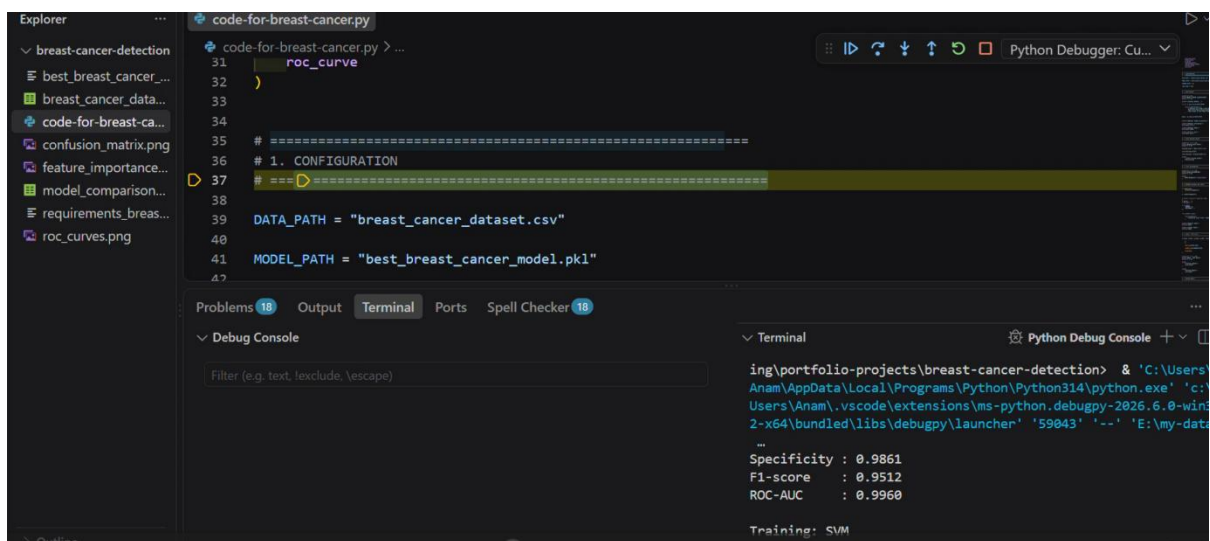
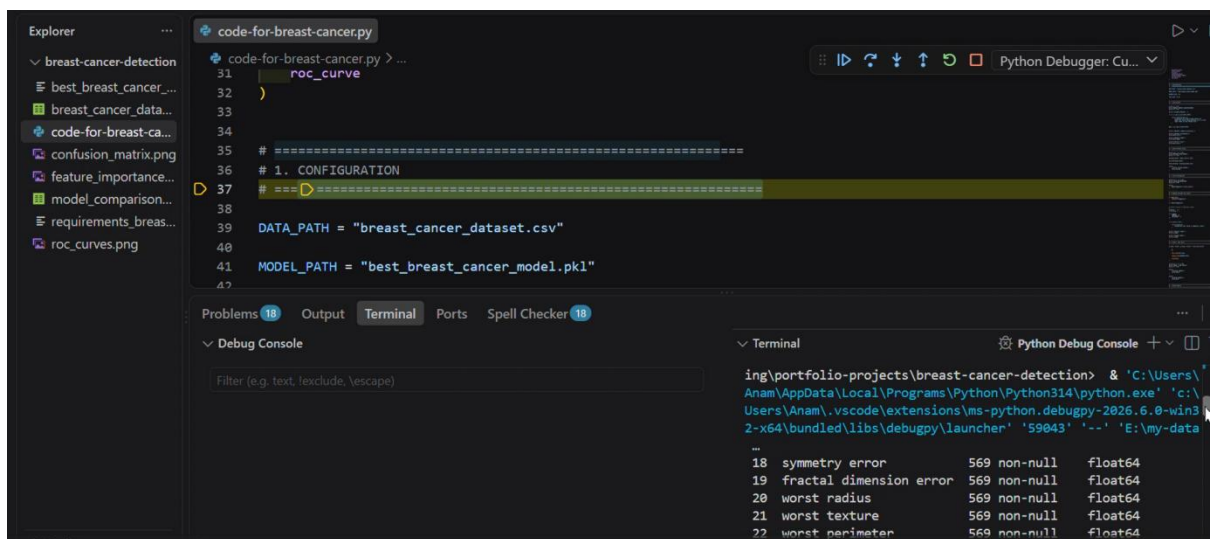
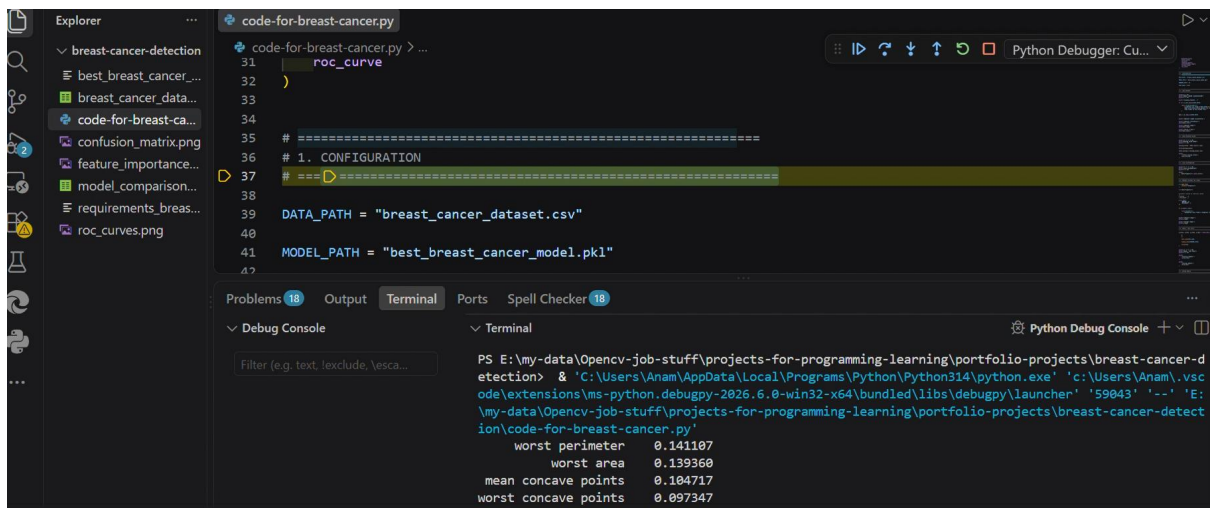


```
=====
# 1. CONFIGURATION
# ===>=====
DATA_PATH = "breast_cancer_dataset.csv"
MODEL_PATH = "best_breast_cancer_model.pkl"
=====
```

	Model	Accuracy	Precision	Recall	Sensitivity	Specificity	F1-score	ROC-AUC
	Random Forest	0.973684	1.000000	0.928571	0.928571	1.000000	0.962963	0.998016
	Logistic Regression	0.964912	0.975000	0.928571	0.928571	0.986111	0.951220	0.996032

```
worst perimeter      0.141107
worst area          0.139360
mean concave points 0.104717
worst concave points 0.097347
```

```
Logistic Regression 0.964912 0.975000 0.928571 0.928571 0.986111 0.951220 0.996032
SVM                 0.973684 1.000000 0.928571 0.928571 1.000000 0.962963 0.994709
KNN                 0.956140 0.974359 0.904762 0.904762 0.986111 0.938272 0.982308
Decision Tree       0.903509 0.897436 0.833333 0.833333 0.944444 0.864198 0.888889
```




```
code-for-breast-cancer.py
31 roc_curve
32 )
33
34
35 # =====
36 # 1. CONFIGURATION
37 # =====
38
39 DATA_PATH = "breast_cancer_dataset.csv"
40
41 MODEL_PATH = "best_breast_cancer_model.pkl"
42
```

```
PS E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection> & 'C:\Users\Anam\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\Anam\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '59043' '--' 'E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection\code-for-breast-cancer.py'
mean area    0.838420

=====
PROJECT COMPLETED
=====
```

```
code-for-breast-cancer.py
31 roc_curve
32 )
33
34
35 # =====
36 # 1. CONFIGURATION
37 # =====
38
39 DATA_PATH = "breast_cancer_dataset.csv"
40
41 MODEL_PATH = "best_breast_cancer_model.pkl"
42
```

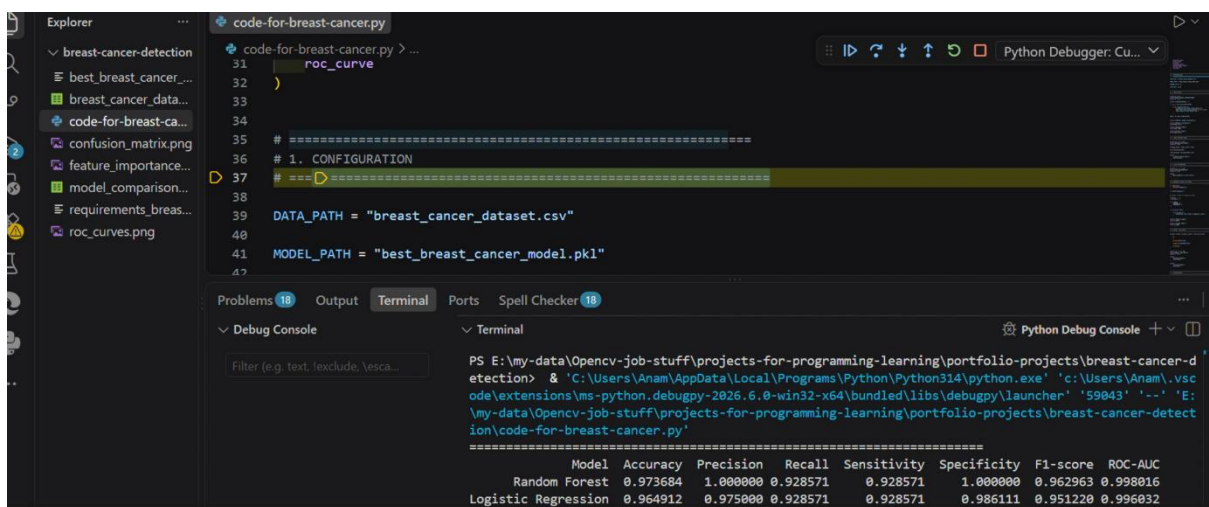
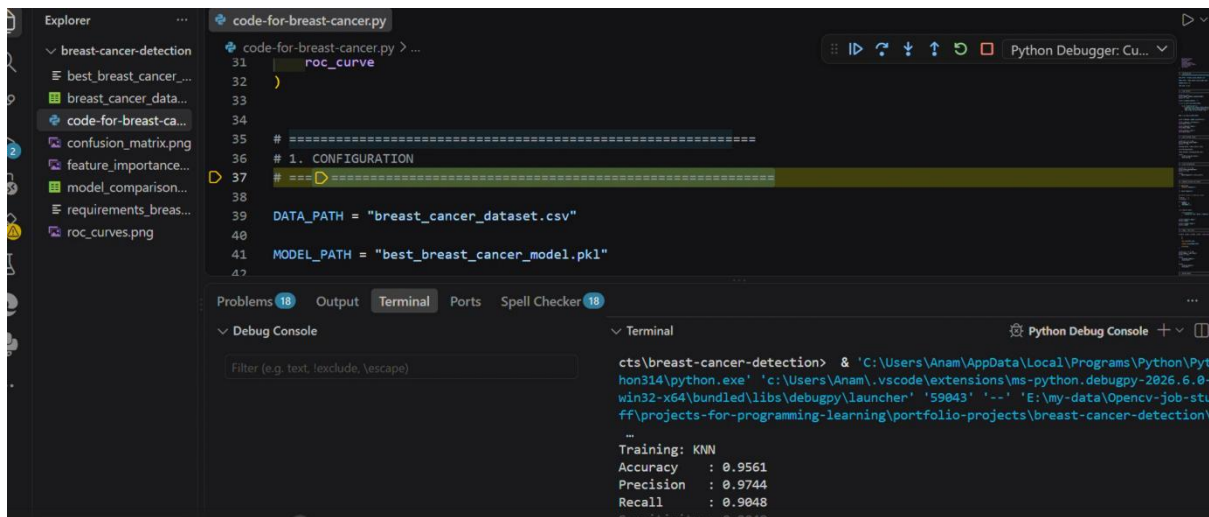
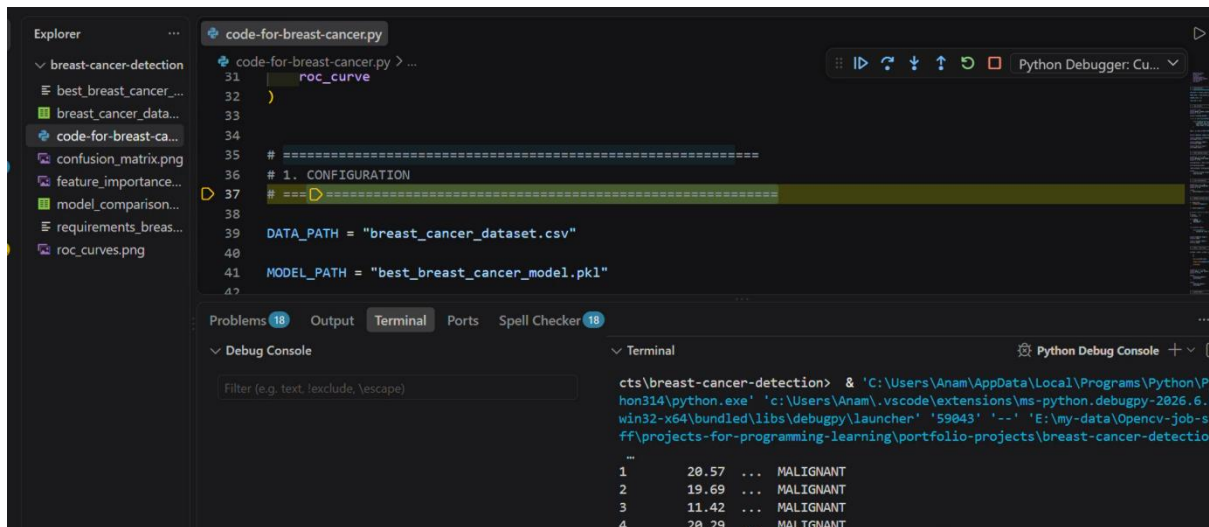
```
PS E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection> & 'C:\Users\Anam\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\Anam\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '59043' '--' 'E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection\code-for-breast-cancer.py'
mean area    0.838420

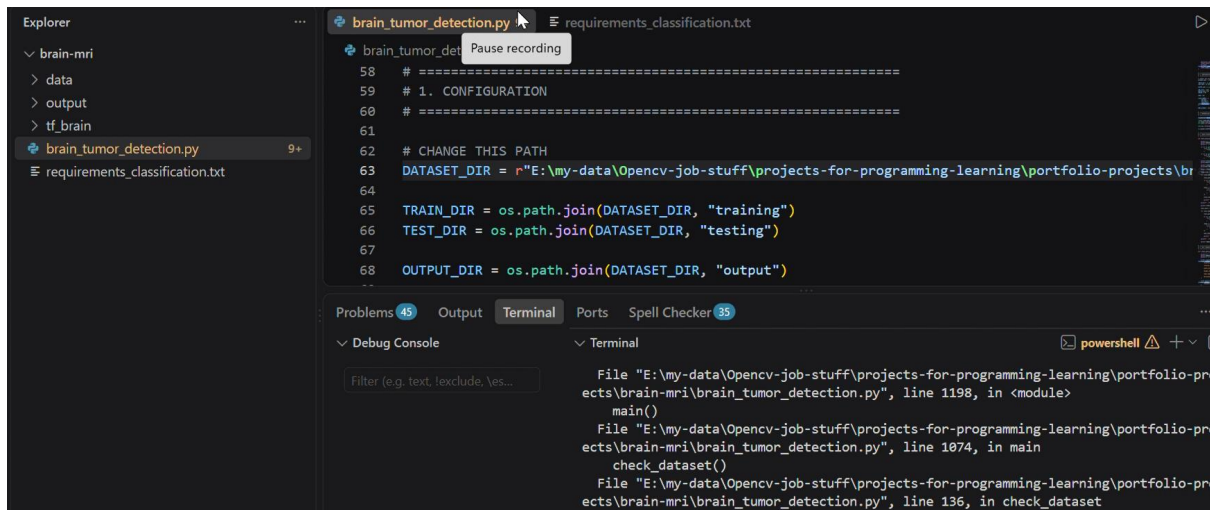
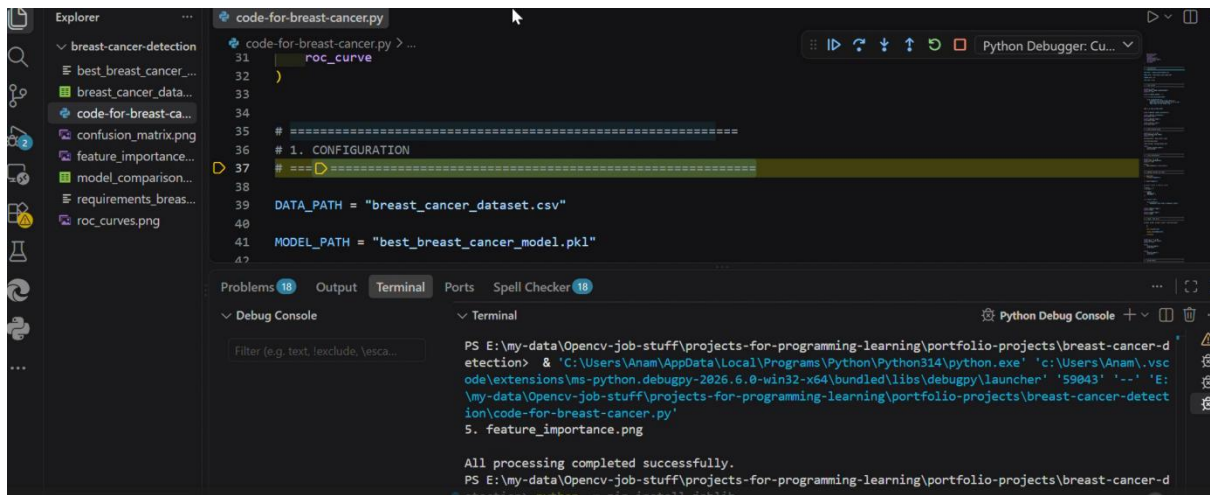
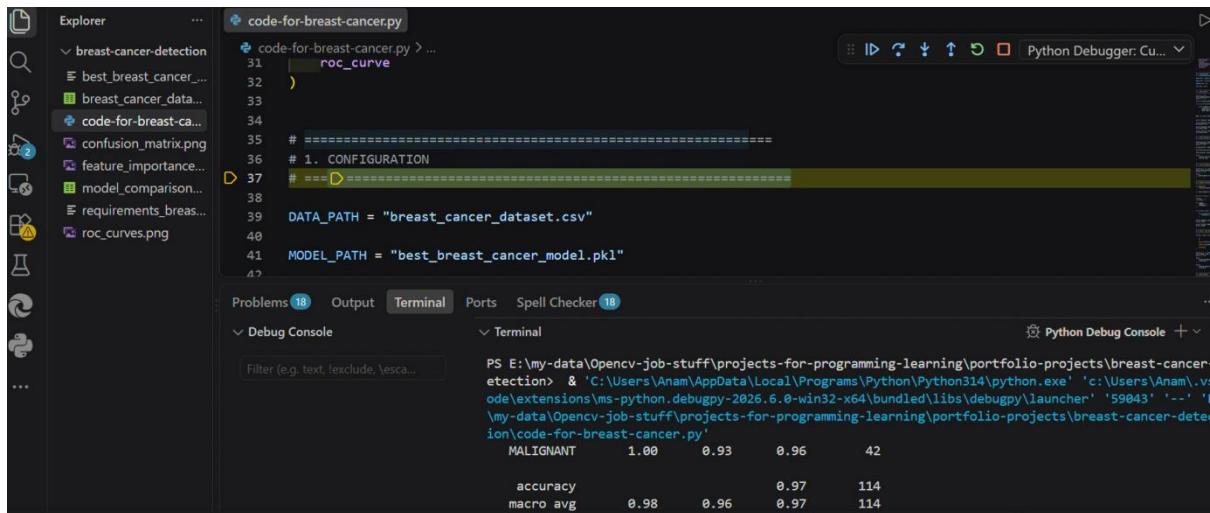
=====
PROJECT COMPLETED
=====
```

```
code-for-breast-cancer.py
31 roc_curve
32 )
33
34
35 # =====
36 # 1. CONFIGURATION
37 # =====
38
39 DATA_PATH = "breast_cancer_dataset.csv"
40
41 MODEL_PATH = "best_breast_cancer_model.pkl"
42
```

```
PS E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection> & 'C:\Users\Anam\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\Anam\vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '59043' '--' 'E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\breast-cancer-detection\code-for-breast-cancer.py'
mean area    0.838420

=====
PROJECT COMPLETED
=====
```





This screenshot shows the first two epochs of training. The code in `brain_tumor_detection.py` defines four classes: `glioma`, `notumor`, `pituitary`, and `meningioma`. The terminal output shows the progress of the training process.

```
76 CLASS_NAMES = [  
77     "glioma",  
78     "notumor", "notumor": Unknown word.  
79     "pituitary",  
80     "meningioma" "meningioma": Unknown word.  
81 ]  
82  
83 NUM_CLASSES = len(CLASS_NAMES)  
84  
85  
86 # =====
```

Terminal Output:

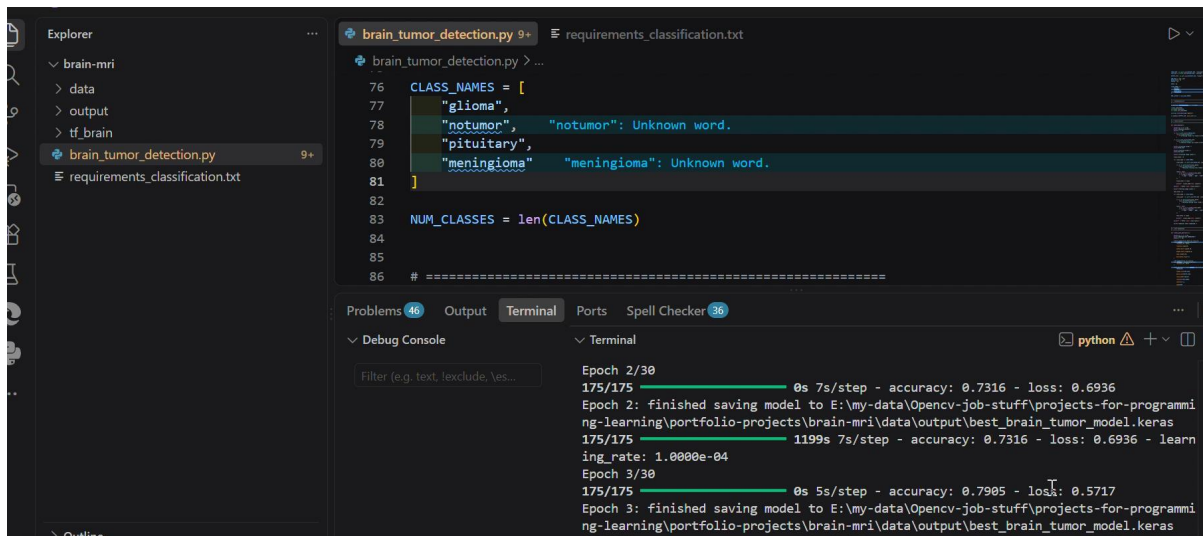
```
Epoch 2/30  
175/175 ----- 0s 7s/step - accuracy: 0.7316 - loss: 0.6936  
Epoch 2: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programmi  
ng-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras  
175/175 ----- 1199s 7s/step - accuracy: 0.7316 - loss: 0.6936 - learn  
ing_rate: 1.0000e-04  
Epoch 3/30  
175/175 ----- 0s 5s/step - accuracy: 0.7905 - loss: 0.5717  
Epoch 3: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programmi  
ng-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras  
175/175 ----- 941s 5s/step - accuracy: 0.7905 - loss: 0.5717 - learn
```

This screenshot shows the third epoch of training. The terminal output continues from the previous epoch, showing the completion of epoch 3 and the saving of the model.

```
Epoch 3/30  
175/175 ----- 0s 5s/step - accuracy: 0.7905 - loss: 0.5717  
Epoch 3: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programmi  
ng-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
```

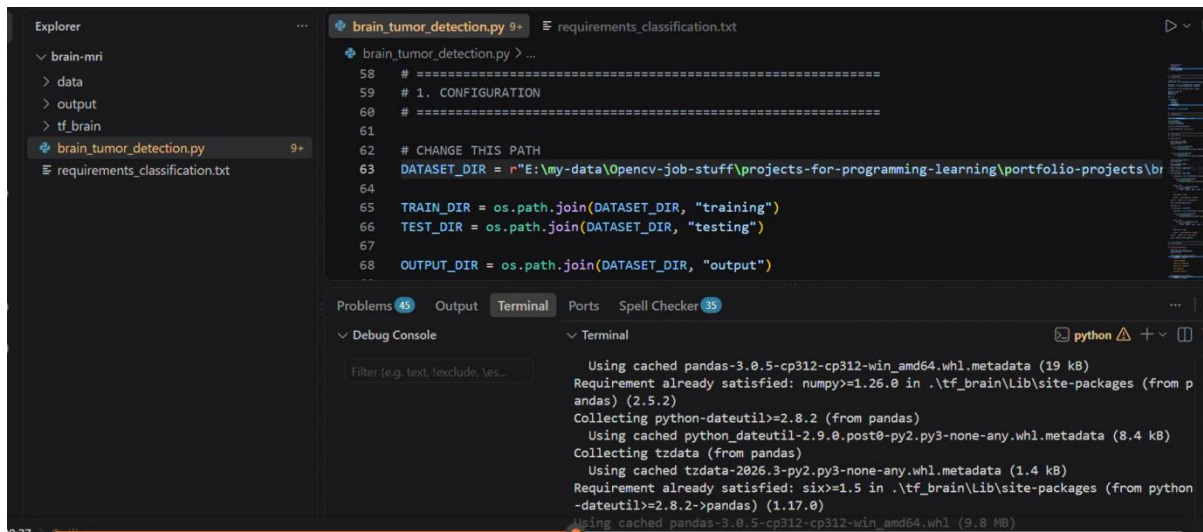
This screenshot shows the dataset check and class mapping. The terminal output indicates that the dataset check is completed and provides the class mapping for the four classes.

```
Dataset check completed.  
  
===== CREATING DATA GENERATORS =====  
Found 5600 images belonging to 4 classes.  
Found 1600 images belonging to 4 classes.  
  
Class mapping:  
{'glioma': 0, 'notumor': 1, 'pituitary': 2, 'meningioma': 3}
```



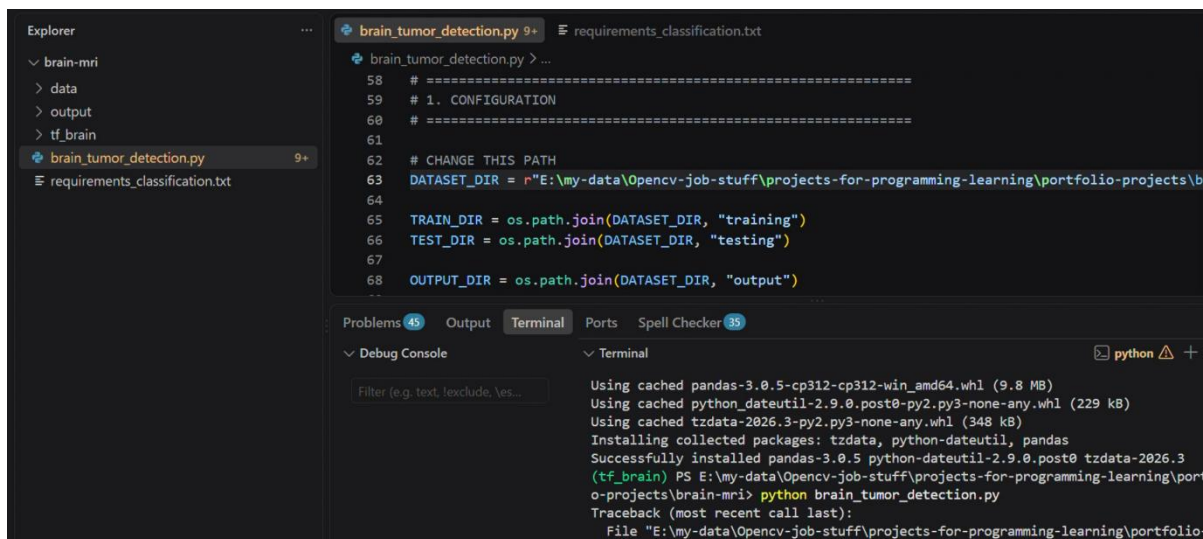
```
76 CLASS_NAMES = [
77     "glioma",
78     "notumor", "notumor": Unknown word.
79     "pituitary",
80     "meningioma" "meningioma": Unknown word.
81 ]
82
83 NUM_CLASSES = len(CLASS_NAMES)
84
85
86 # =====
```

Epoch 2/30
175/175 ██████████ 0s 7s/step - accuracy: 0.7316 - loss: 0.6936
Epoch 2: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
175/175 ██████████ 1199s 7s/step - accuracy: 0.7316 - loss: 0.6936 - learning_rate: 1.0000e-04
Epoch 3/30
175/175 ██████████ 0s 5s/step - accuracy: 0.7905 - loss: 0.5717
Epoch 3: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras



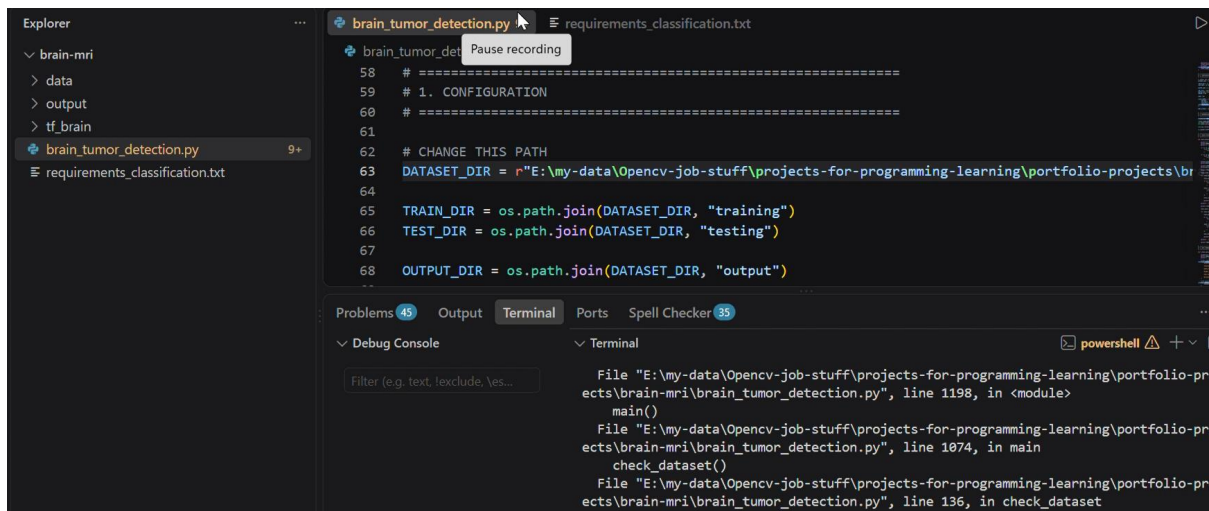
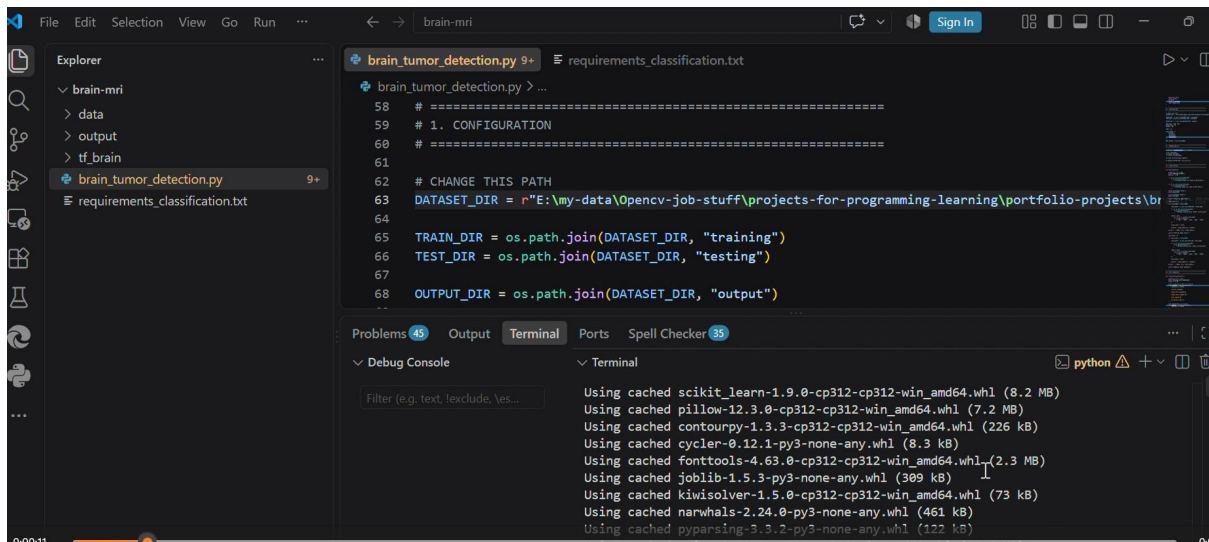
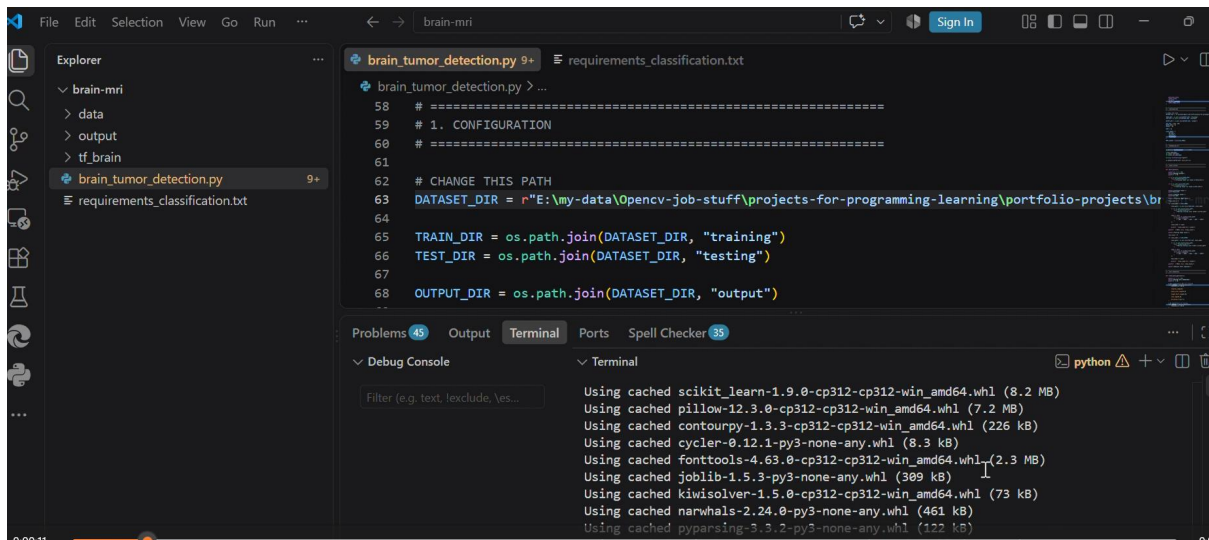
```
58 # =====
59 # 1. CONFIGURATION
60 # =====
61
62 # CHANGE THIS PATH
63 DATASET_DIR = r"E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data"
64
65 TRAIN_DIR = os.path.join(DATASET_DIR, "training")
66 TEST_DIR = os.path.join(DATASET_DIR, "testing")
67
68 OUTPUT_DIR = os.path.join(DATASET_DIR, "output")
69
```

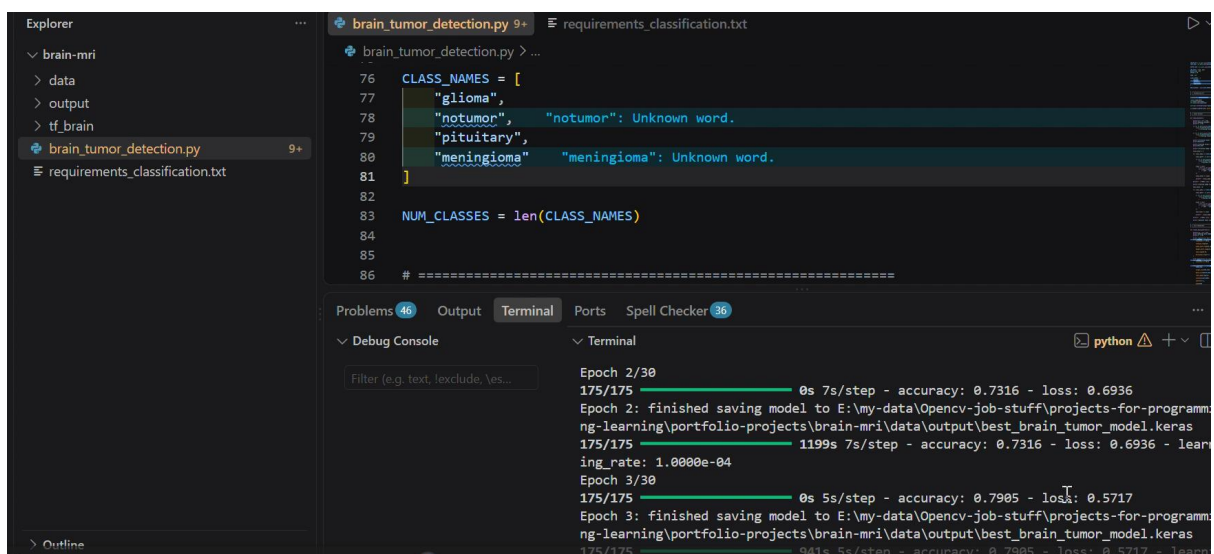
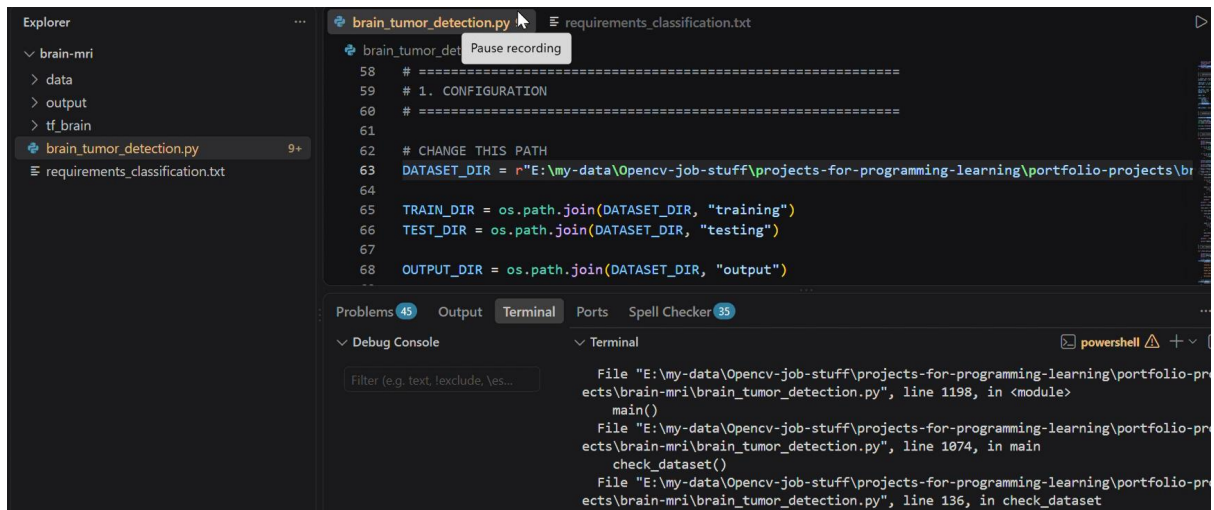
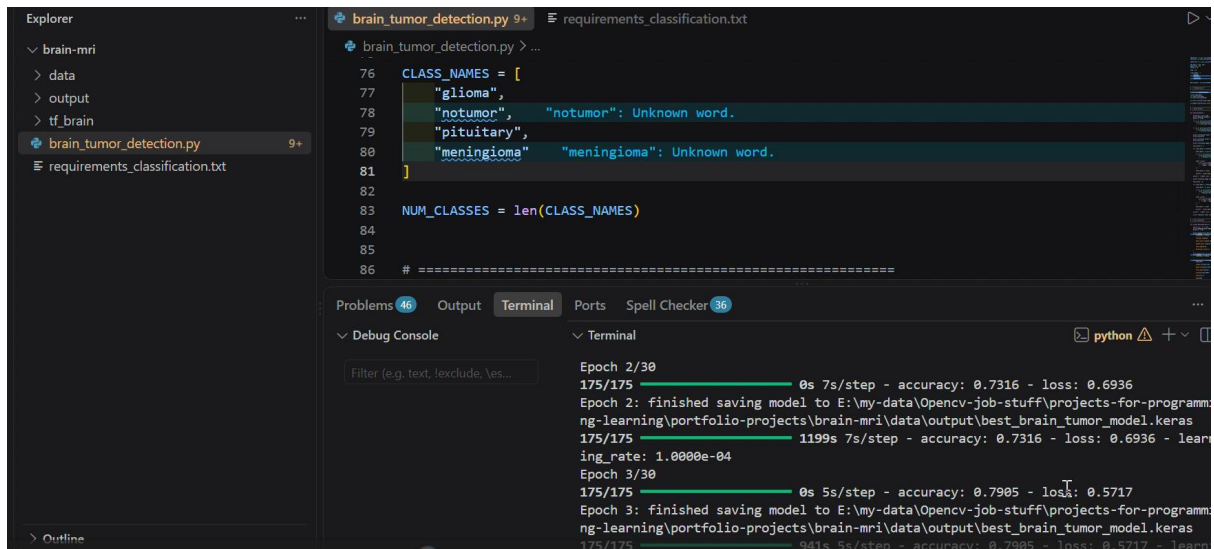
Using cached pandas-3.0.5-cp312-cp312-win_amd64.whl.metadata (19 kB)
Requirement already satisfied: numpy>=1.26.0 in .\tf_brain\Lib\site-packages (from pandas) (2.5.2)
Collecting python-dateutil>=2.8.2 (from pandas)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
Using cached tzdata-2026.3-py2.py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: six>=1.5 in .\tf_brain\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Using cached pandas-3.0.5-cp312-cp312-win_amd64.whl (9.8 MB)



```
58 # =====
59 # 1. CONFIGURATION
60 # =====
61
62 # CHANGE THIS PATH
63 DATASET_DIR = r"E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data"
64
65 TRAIN_DIR = os.path.join(DATASET_DIR, "training")
66 TEST_DIR = os.path.join(DATASET_DIR, "testing")
67
68 OUTPUT_DIR = os.path.join(DATASET_DIR, "output")
69
```

Using cached pandas-3.0.5-cp312-cp312-win_amd64.whl (9.8 MB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached tzdata-2026.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, python-dateutil, pandas
Successfully installed pandas-3.0.5 python-dateutil-2.9.0.post0 tzdata-2026.3
(tf_brain) PS E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri> python brain_tumor_detection.py
Traceback (most recent call last):
File "E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-





The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure: `brain-mri` (containing `data`, `output`, and `tf_brain`), `brain_tumor_detection.py` (selected), and `requirements_classification.txt`. The main editor shows the `brain_tumor_detection.py` file with the following code:

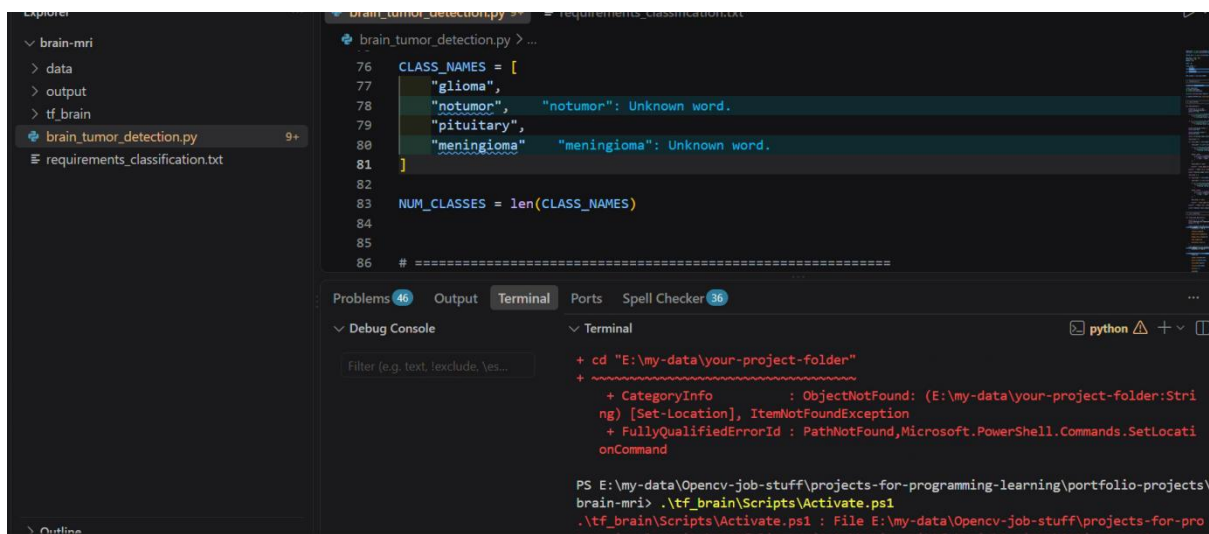
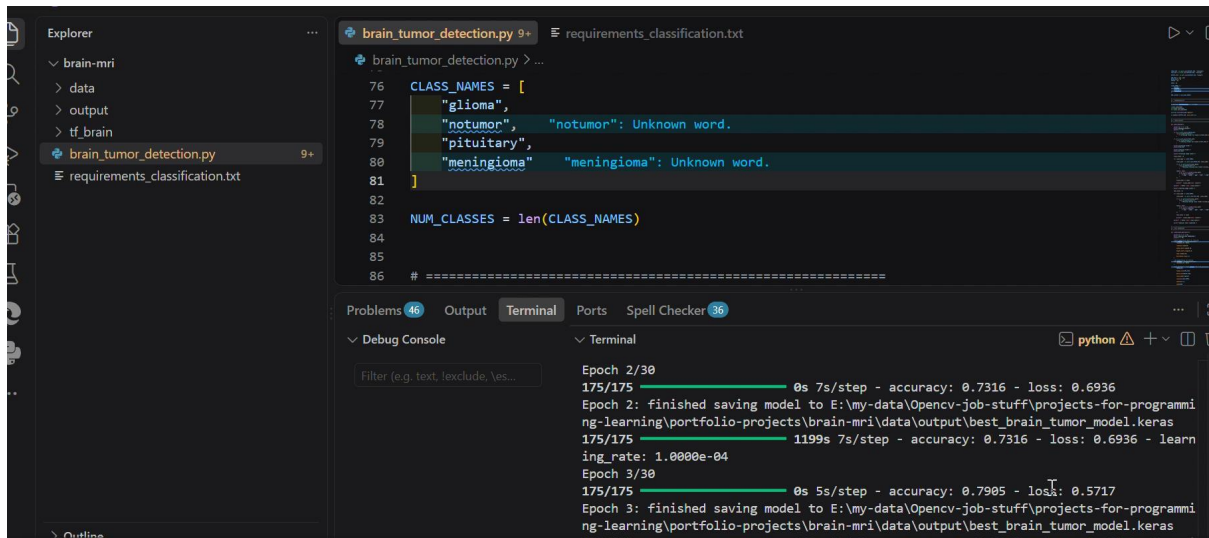
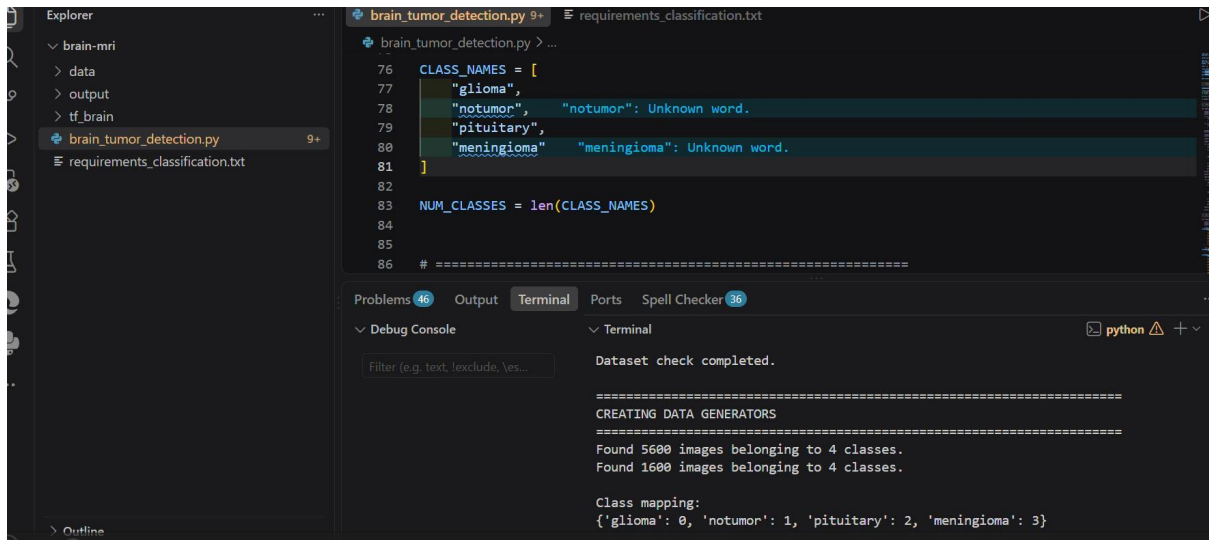
```
76 CLASS_NAMES = [  
77     "glioma",  
78     "notumor", "notumor": Unknown word.  
79     "pituitary",  
80     "meningioma" "meningioma": Unknown word.  
81 ]  
82  
83 NUM_CLASSES = len(CLASS_NAMES)  
84  
85  
86 # =====
```

The bottom panel shows the `Terminal` tab with the following output:

```
Epoch 2/30  
175/175 ----- 0s 7s/step - accuracy: 0.7316 - loss: 0.6936  
Epoch 2: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programmi  
ng-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras  
175/175 ----- 1199s 7s/step - accuracy: 0.7316 - loss: 0.6936 - learn  
ing_rate: 1.0000e-04  
Epoch 3/30  
175/175 ----- 0s 5s/step - accuracy: 0.7905 - loss: 0.5717  
Epoch 3: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programmi  
ng-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras  
175/175 ----- 941s 5s/step - accuracy: 0.7905 - loss: 0.5717 - learn
```

This screenshot is identical to the one above, showing the same code and terminal output in the Visual Studio Code interface.

This screenshot is also identical to the previous ones, displaying the same code and terminal output within the VS Code environment.



```
76 CLASS_NAMES = [  
77     "glioma",  
78     "notumor", "notumor": Unknown word.  
79     "pituitary",  
80     "meningioma" "meningioma": Unknown word.  
81 ]  
82  
83 NUM_CLASSES = len(CLASS_NAMES)  
84  
85  
86 # =====
```

Epoch 2/30
175/175 ————— 0s 7s/step - accuracy: 0.7316 - loss: 0.6936
Epoch 2: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
175/175 ————— 1199s 7s/step - accuracy: 0.7316 - loss: 0.6936 - learning_rate: 1.0000e-04
Epoch 3/30
175/175 ————— 0s 5s/step - accuracy: 0.7905 - loss: 0.5717
Epoch 3: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras

```
76 CLASS_NAMES = [  
77     "glioma",  
78     "notumor", "notumor": Unknown word.  
79     "pituitary",  
80     "meningioma" "meningioma": Unknown word.  
81 ]  
82  
83 NUM_CLASSES = len(CLASS_NAMES)  
84  
85  
86 # =====
```

Dataset check completed.

=====

CREATING DATA GENERATORS
=====

Found 5600 images belonging to 4 classes.
Found 1600 images belonging to 4 classes.

Class mapping:
{'glioma': 0, 'notumor': 1, 'pituitary': 2, 'meningioma': 3}

```
76 CLASS_NAMES = [  
77     "glioma",  
78     "notumor", "notumor": Unknown word.  
79     "pituitary",  
80     "meningioma" "meningioma": Unknown word.  
81 ]  
82  
83 NUM_CLASSES = len(CLASS_NAMES)  
84  
85  
86 # =====
```

in-mri\data\training

Testing folder:
E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\testing

Training image counts:
glioma : 1400
Traceback (most recent call last):

```
76 CLASS_NAMES = [
77     "glioma",
78     "notumor", "notumor": Unknown word.
79     "pituitary",
80     "meningioma" "meningioma": Unknown word.
81 ]
82
83 NUM_CLASSES = len(CLASS_NAMES)
84
85
86 # =====
```

Class mapping:
{'glioma': 0, 'notumor': 1, 'pituitary': 2, 'meningioma': 3}

Training samples: 5600
Testing samples: 1600

=====

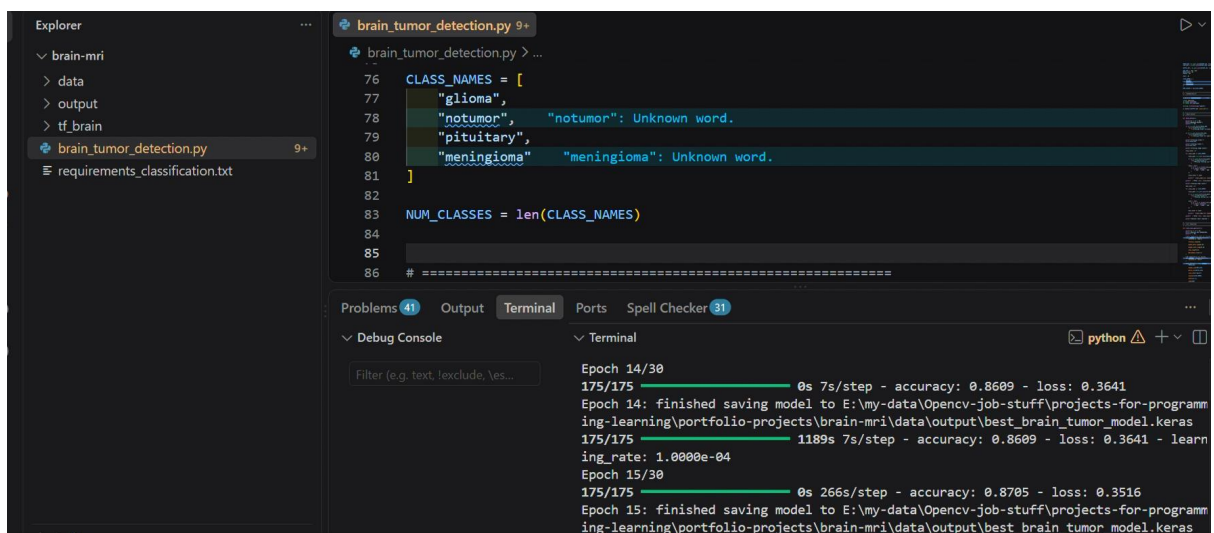
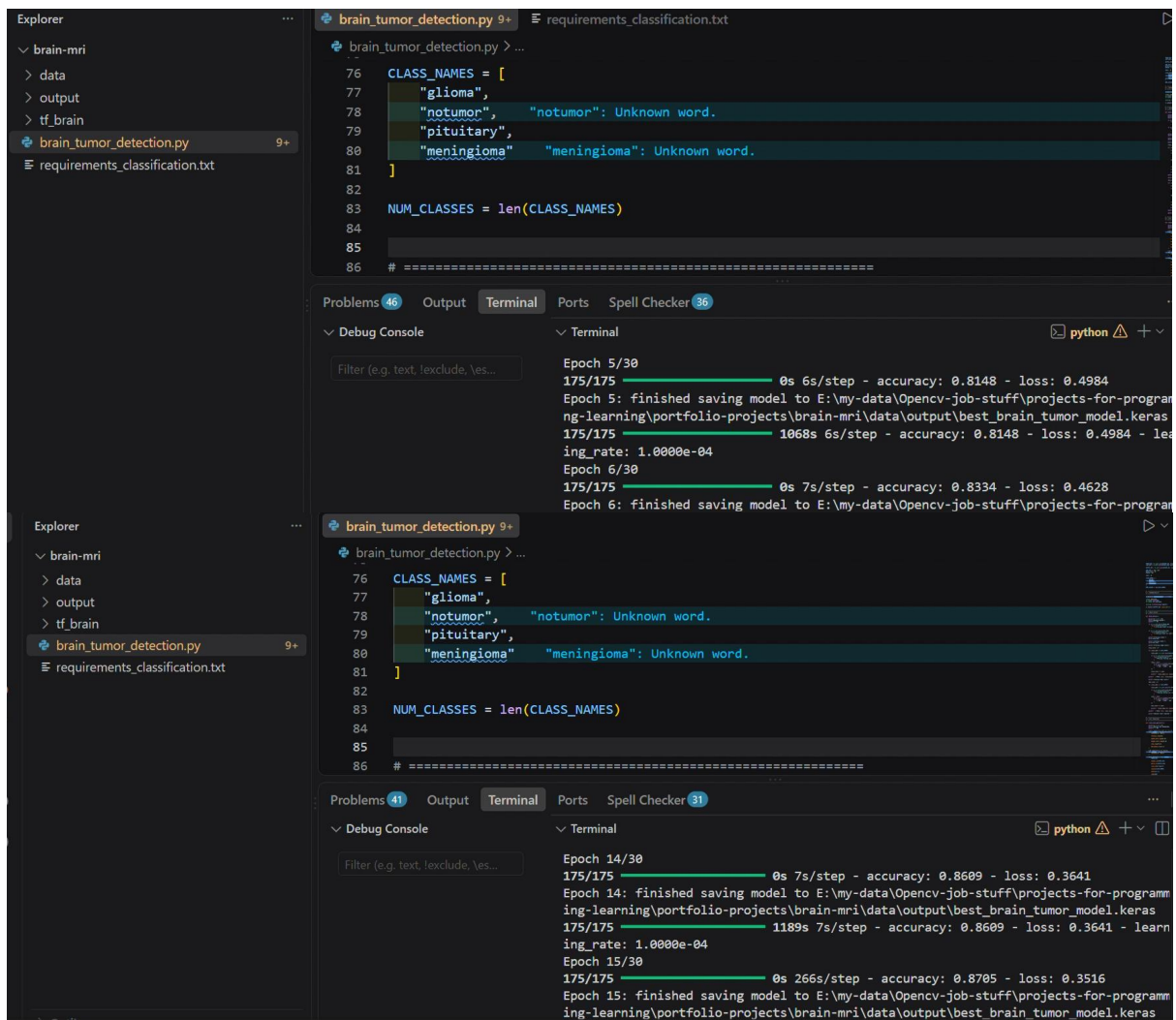
BUILDING CNN MODEL

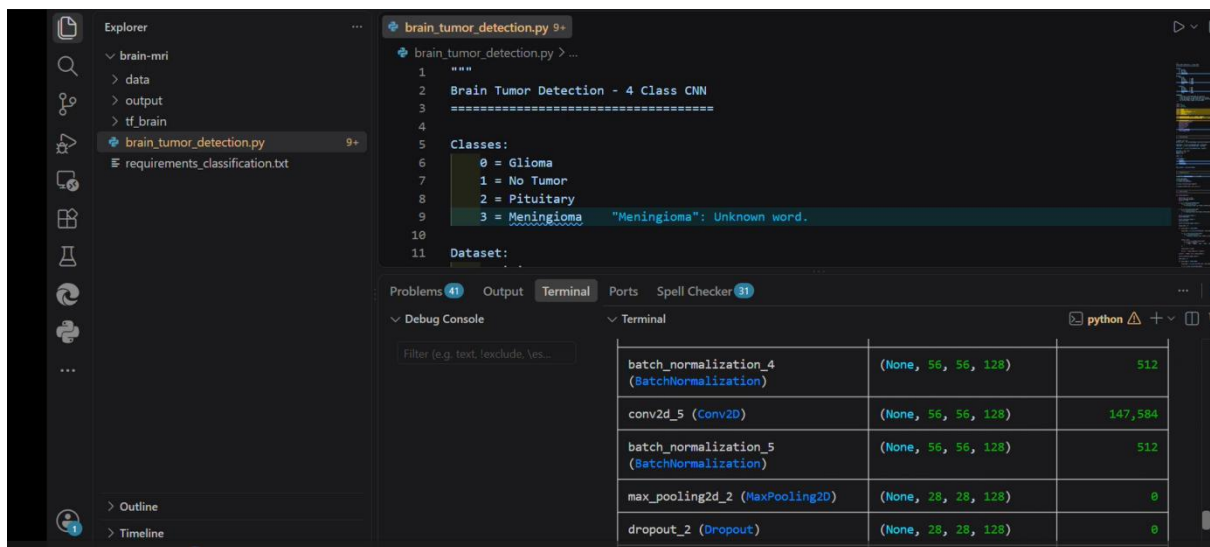
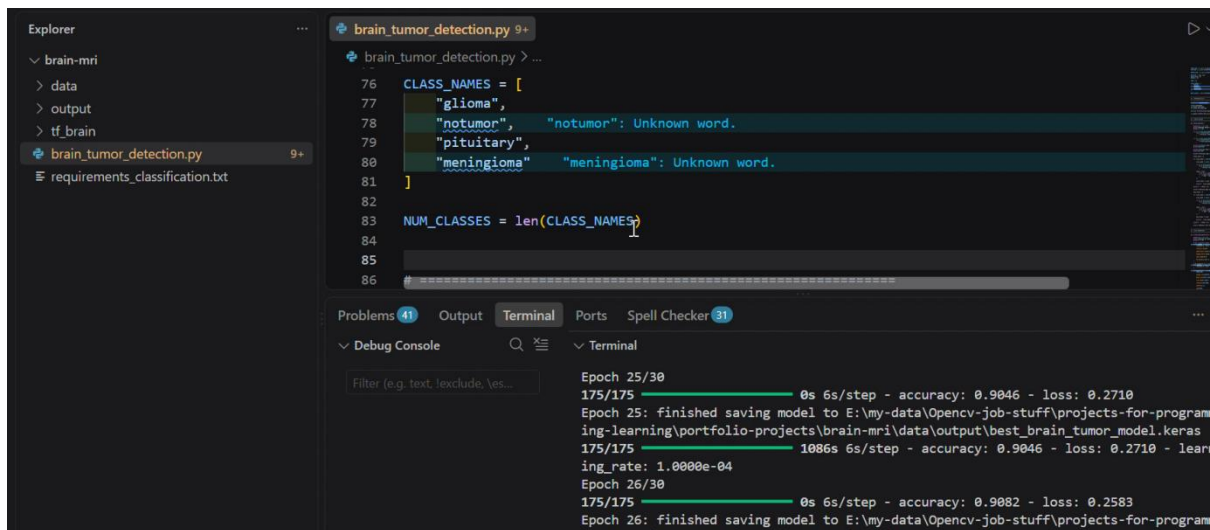
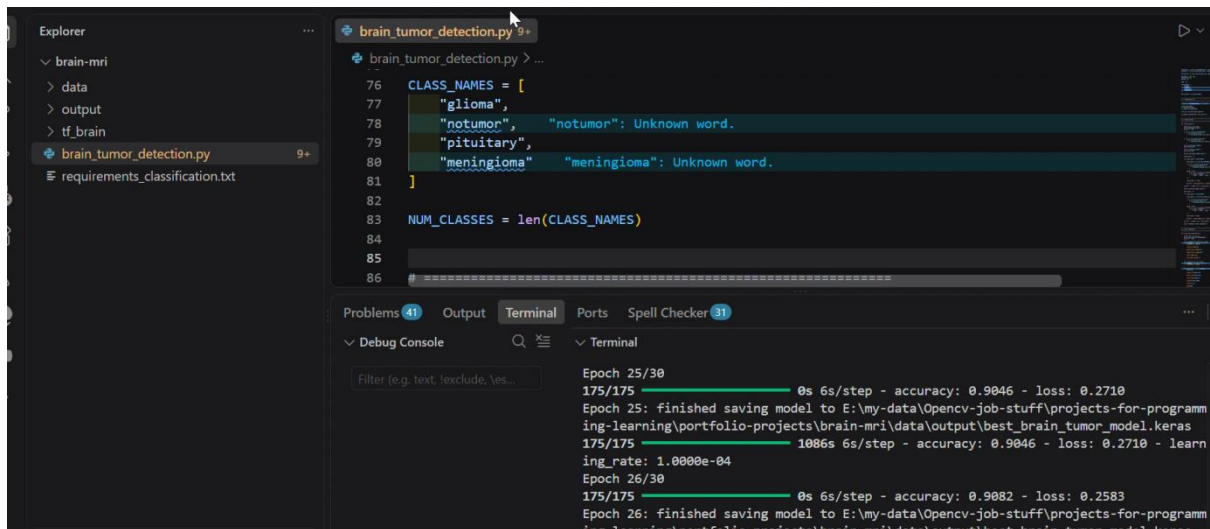
=====

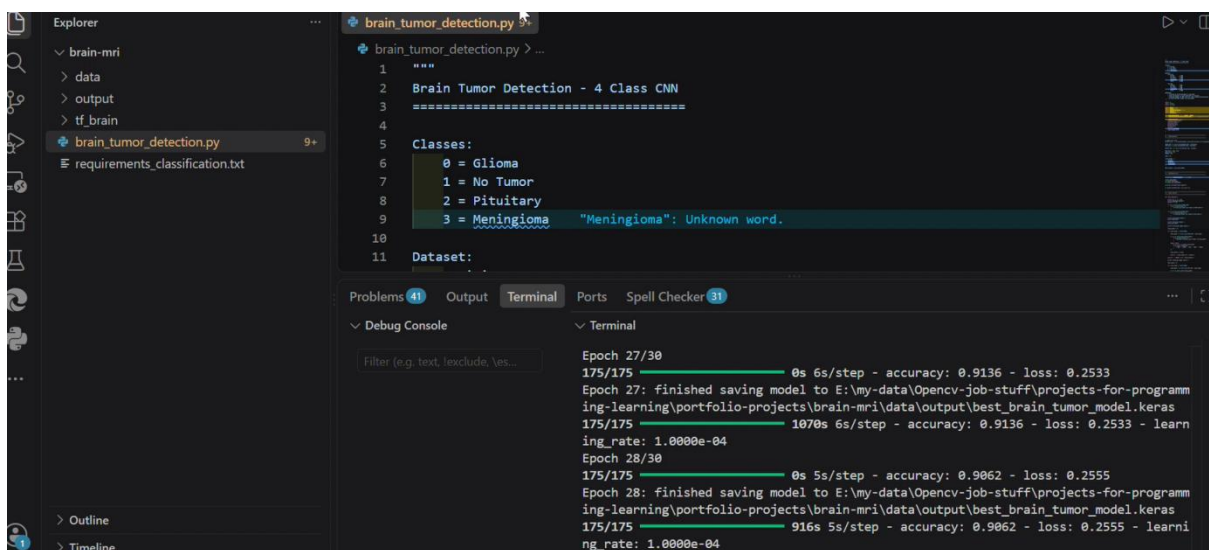
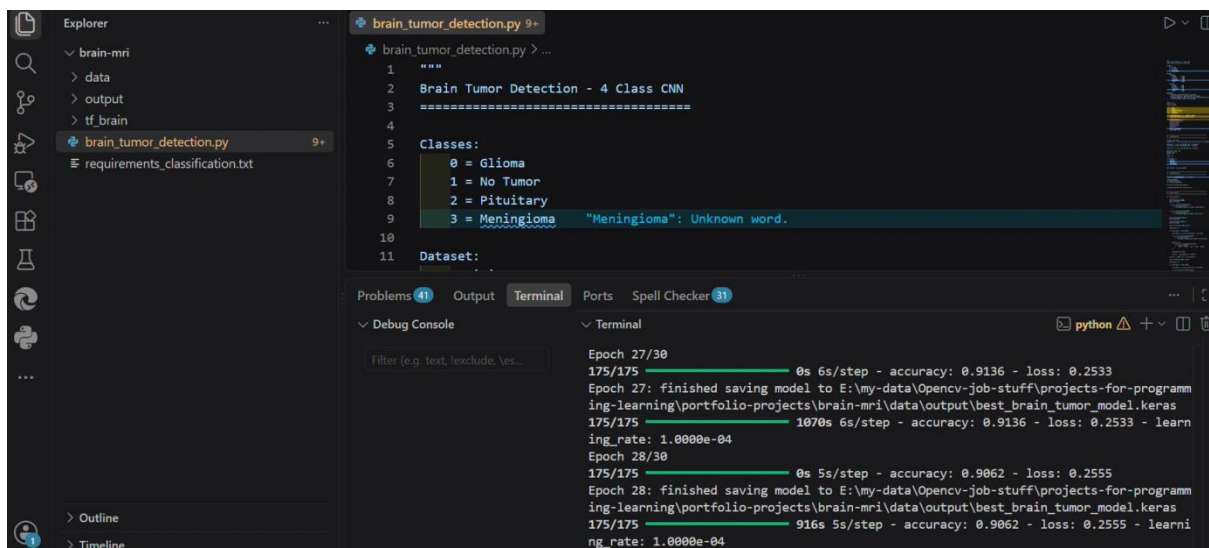
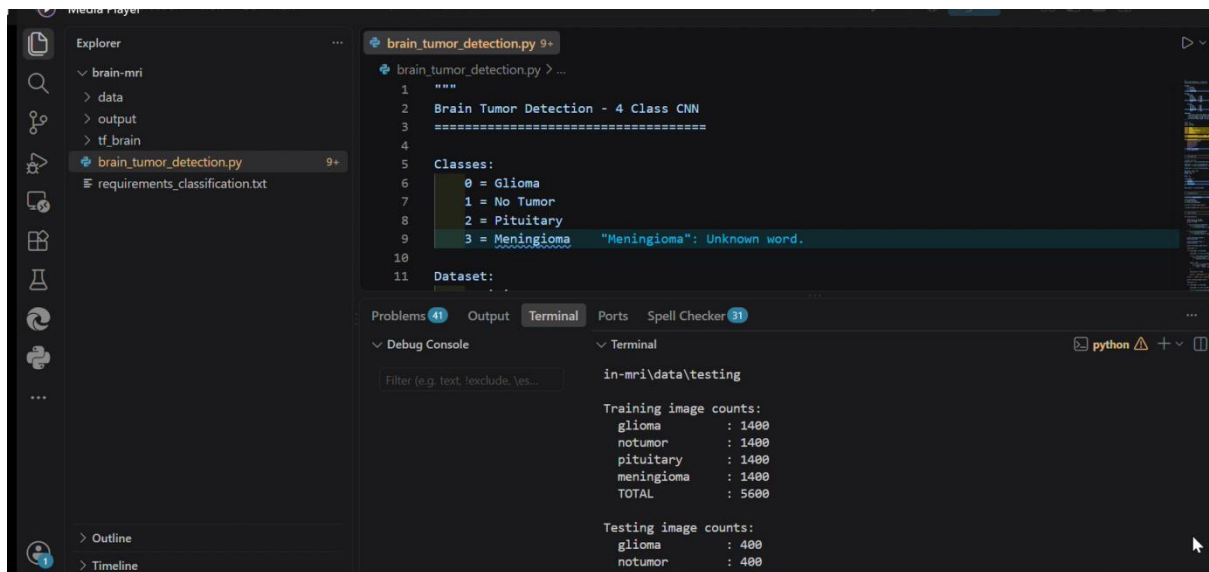
10000 00:00:1786889123.790192 11204 cpu_feature_guard.cc:227] This TensorFlow bina

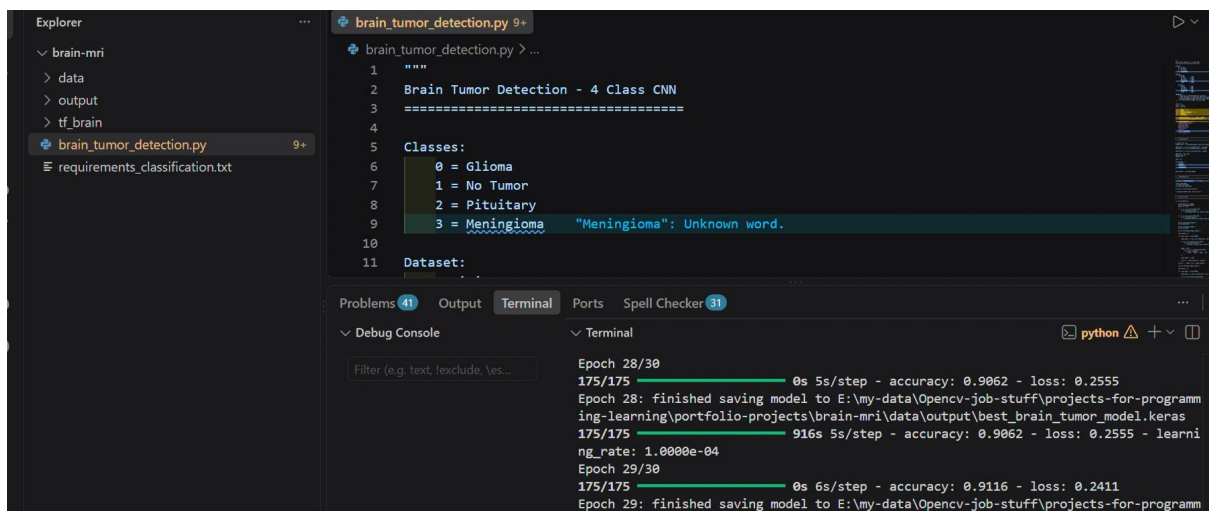
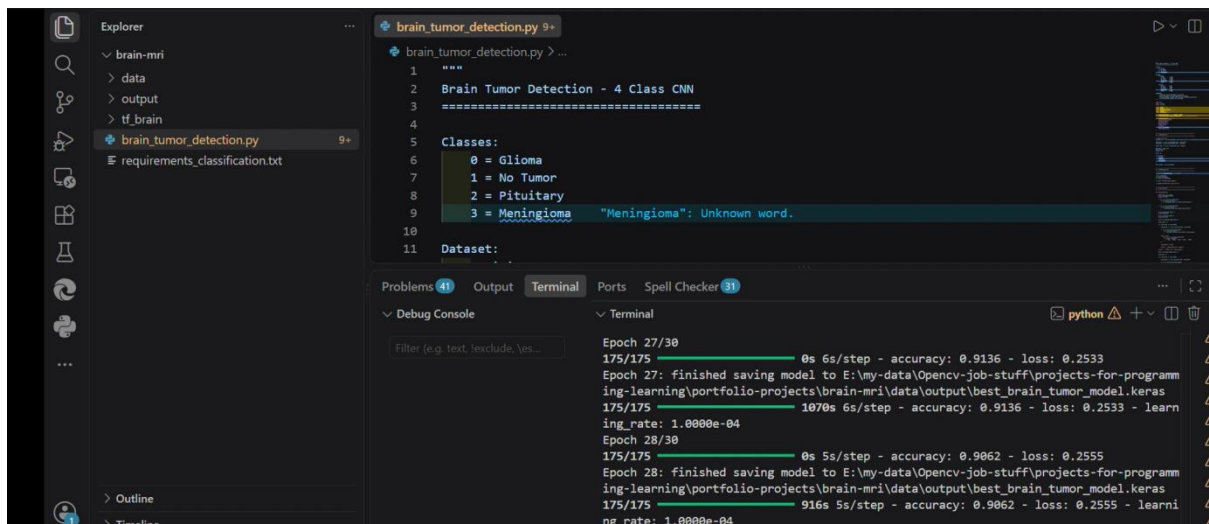
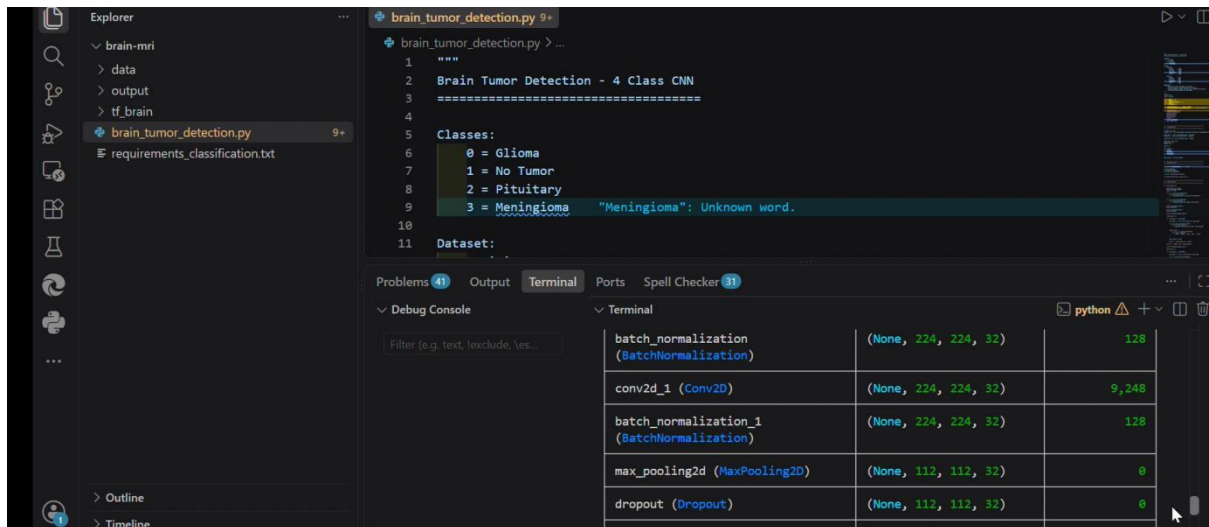
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout_2 (Dropout)	(None, 28, 28, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0

Epoch 5/30
175/175 ██████████ 0s 6s/step - accuracy: 0.8148 - loss: 0.4984
Epoch 5: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
175/175 ██████████ 1068s 6s/step - accuracy: 0.8148 - loss: 0.4984 - learning_rate: 1.0000e-04
Epoch 6/30
175/175 ██████████ 0s 7s/step - accuracy: 0.8334 - loss: 0.4628
Epoch 6: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras









The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, output, tf_brain, brain_tumor_detection.py (9+), and requirements_classification.txt. The main editor displays the brain_tumor_detection.py file with the following code:

```
1 """
2 Brain Tumor Detection - 4 Class CNN
3 =====
4
5 Classes:
6 0 = Glioma
7 1 = No Tumor
8 2 = Pituitary
9 3 = Meningioma "Meningioma": Unknown word.
10
11 Dataset:
```

The terminal output shows the training progress:

```
Epoch 28/30
175/175 0s 5s/step - accuracy: 0.9062 - loss: 0.2555
Epoch 28: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programm
ing-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
175/175 916s 5s/step - accuracy: 0.9062 - loss: 0.2555 - learni
ng_rate: 1.0000e-04
Epoch 29/30
175/175 0s 6s/step - accuracy: 0.9116 - loss: 0.2411
Epoch 29: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programm
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, output, tf_brain, brain_tumor_detection.py (9+), and requirements_classification.txt. The main editor displays the brain_tumor_detection.py file with the following code:

```
1 """
2 Brain Tumor Detection - 4 Class CNN
3 =====
4
5 Classes:
6 0 = Glioma
7 1 = No Tumor
8 2 = Pituitary
9 3 = Meningioma "Meningioma": Unknown word.
10
11 Dataset:
```

The terminal output shows the training progress:

```
Epoch 28/30
175/175 0s 5s/step - accuracy: 0.9062 - loss: 0.2555
Epoch 28: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programm
ing-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras
175/175 916s 5s/step - accuracy: 0.9062 - loss: 0.2555 - learni
ng_rate: 1.0000e-04
Epoch 29/30
175/175 0s 6s/step - accuracy: 0.9116 - loss: 0.2411
Epoch 29: finished saving model to E:\my-data\Opencv-job-stuff\projects-for-programm
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, output, tf_brain, brain_tumor_detection.py (9+), and requirements_classification.txt. The main editor displays the brain_tumor_detection.py file with the following code:

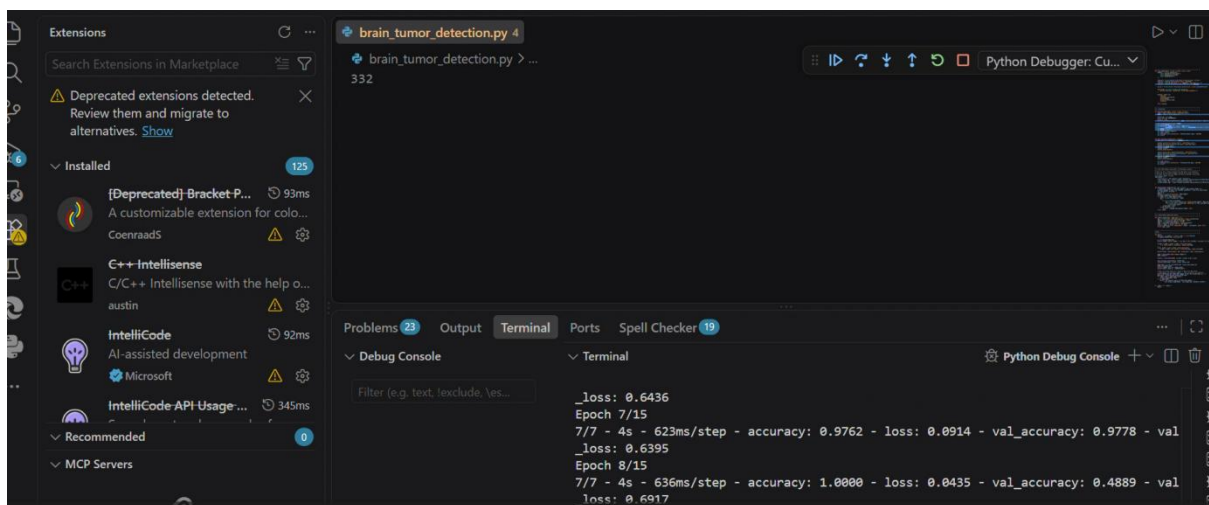
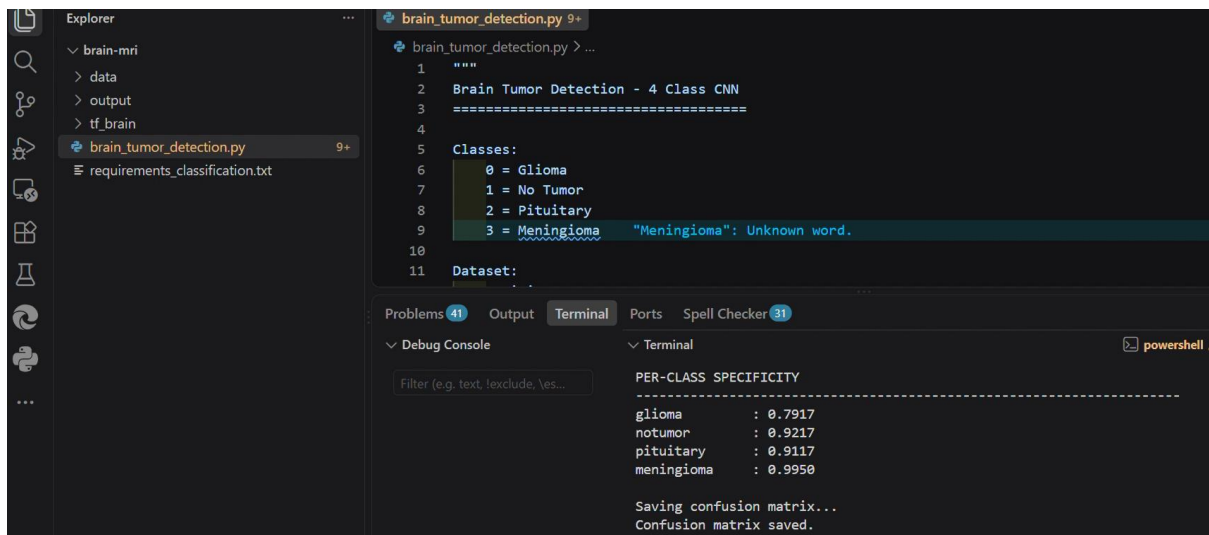
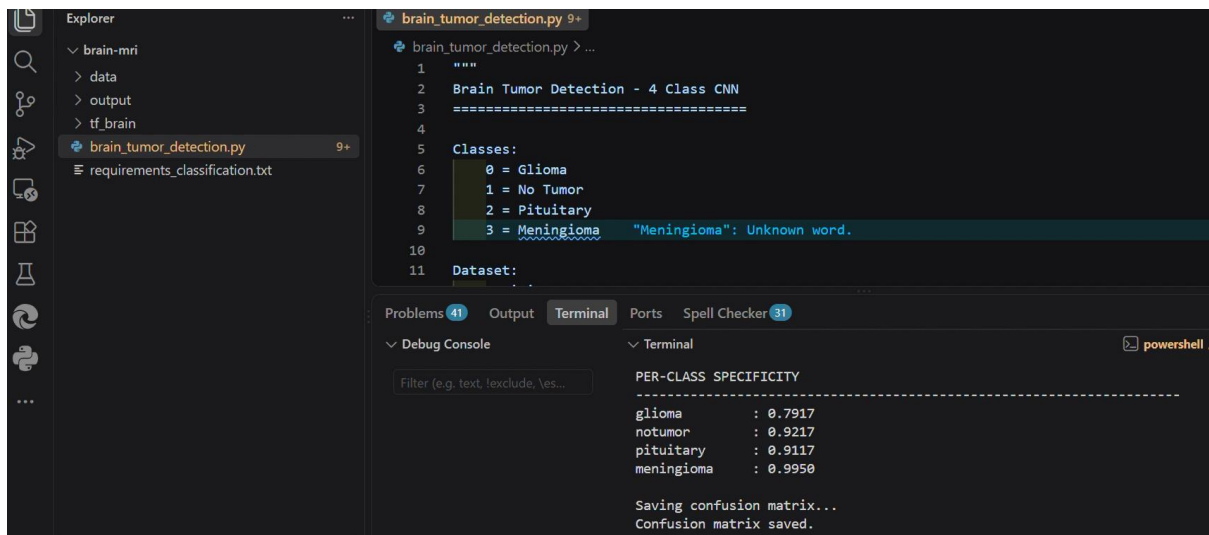
```
1 """
2 Brain Tumor Detection - 4 Class CNN
3 =====
4
5 Classes:
6 0 = Glioma
7 1 = No Tumor
8 2 = Pituitary
9 3 = Meningioma "Meningioma": Unknown word.
10
11 Dataset:
```

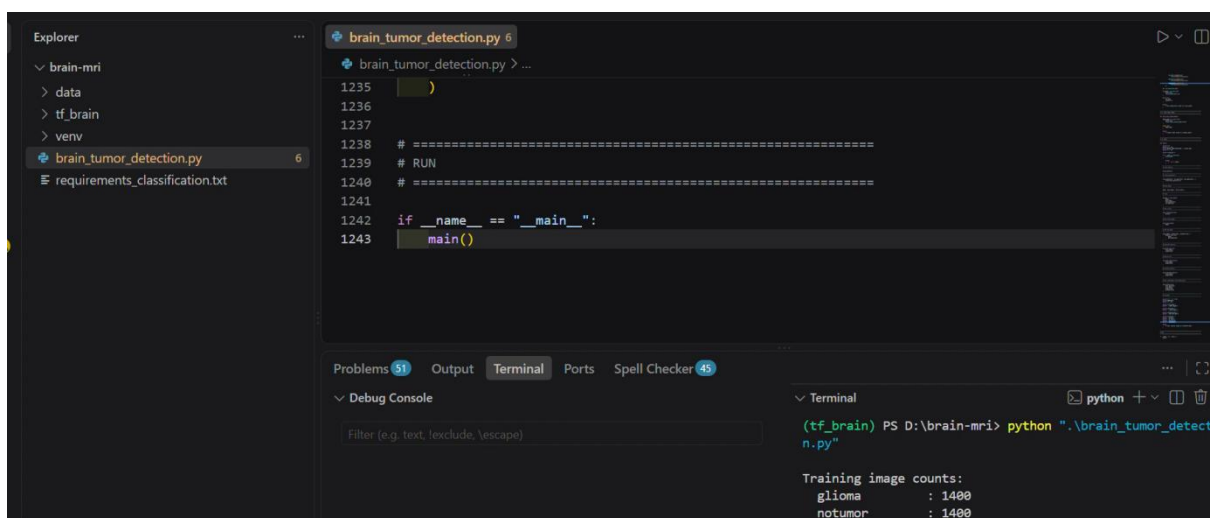
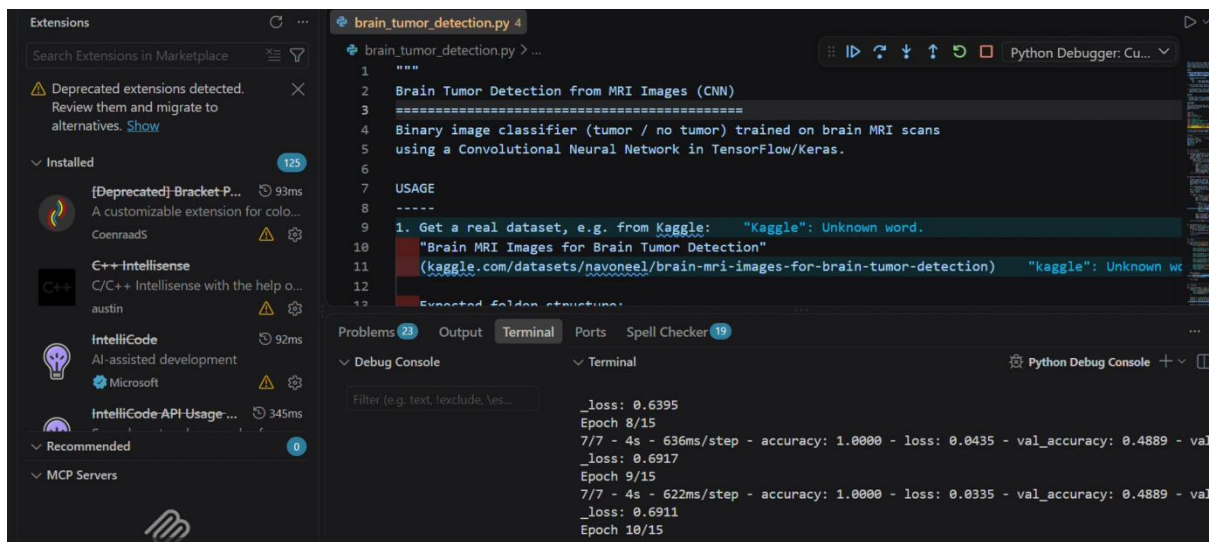
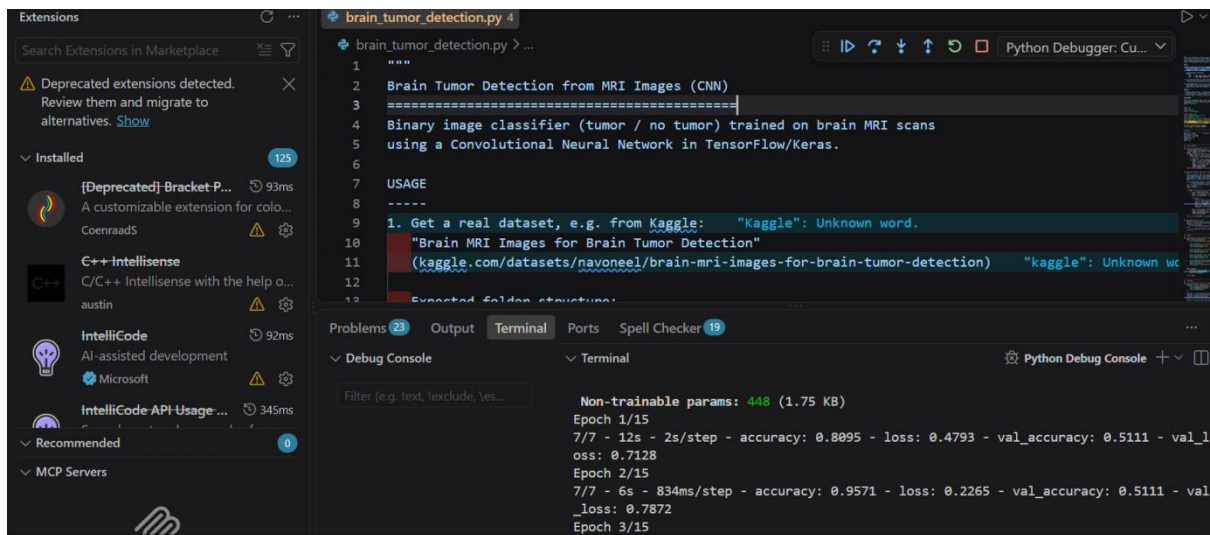
The terminal output shows the class mapping and training details:

```
Class mapping:
{'glioma': 0, 'notumor': 1, 'pituitary': 2, 'meningioma': 3}

Training samples: 5600
Testing samples: 1600

=====
BUILDING CNN MODEL
=====
10000 00:00:1786880123.790182 11304 cpu_feature_guard.cc:227] This TensorFlow bin
```





```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Terminal

(tf_brain) PS D:\brain-mri> python \"brain_tumor_detection.py\"

Step	Time	Accuracy
23/149	3:24 2s/step	accuracy: 0.31
24/149	3:23 2s/step	accuracy: 0.32
25/149	3:22 2s/step	accuracy: 0.32
26/149	3:22 2s/step	accuracy: 0.32

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Terminal

(tf_brain) PS D:\brain-mri> python \"brain_tumor_detection.py\"

Step	Time	Accuracy
46/149	2:03 1s/step	accuracy: 0.73
47/149	2:02 1s/step	accuracy: 0.73
48/149	2:01 1s/step	accuracy: 0.73
49/149	2:01 1s/step	accuracy: 0.73

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Terminal

(tf_brain) PS D:\brain-mri> python \"brain_tumor_detection.py\"

Step	Time	Accuracy
14/149	3:31 2s/step	accuracy: 0.82
15/149	3:29 2s/step	accuracy: 0.82
16/149	3:27 2s/step	accuracy: 0.83
17/149	3:26 2s/step	accuracy: 0.82
18/149	3:24 2s/step	accuracy: 0.82
19/149	3:22 2s/step	accuracy: 0.82

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor displays the `brain_tumor_detection.py` file with line numbers 1235 to 1243. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with the following output:

```
149/149 ██████████ 202s 1s/step - accuracy: 0.81
41 - loss: 0.4983 - val_accuracy: 0.8607 - val_loss: 0.418
6 - learning_rate: 1.0000e-04
Epoch 9/15
1/149 ██████████ 4:16 2s/step - accuracy: 0.84
2/149 ██████████ 3:45 2s/step - accuracy: 0.81
3/149 ██████████ 3:44 2s/step - accuracy: 0.81
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor displays the `brain_tumor_detection.py` file with line numbers 1235 to 1243. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with the following output:

```
43 - loss: 0.5036 - val_accuracy: 0.8607 - val_loss: 0.422
8 - learning_rate: 1.0000e-04
Epoch 10/15
1/149 ██████████ 3:50 2s/step - accuracy: 0.81
2/149 ██████████ 2:54 1s/step - accuracy: 0.81
3/149 ██████████ 2:48 1s/step - accuracy: 0.81
4/149 ██████████ 2:47 1s/step - accuracy: 0.80
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor displays the `brain_tumor_detection.py` file with line numbers 1235 to 1243. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with the following output:

```
149/149 ██████████ 202s 1s/step - accuracy: 0.81
41 - loss: 0.4983 - val_accuracy: 0.8607 - val_loss: 0.418
6 - learning_rate: 1.0000e-04
Epoch 9/15
1/149 ██████████ 4:16 2s/step - accuracy: 0.84
2/149 ██████████ 3:45 2s/step - accuracy: 0.81
3/149 ██████████ 3:44 2s/step - accuracy: 0.81
```

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py
 - requirements_classification.txt

brain_tumor_detection.py

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, \exclude, \escape)

Terminal

	python	+	-	...
9/149	3:16	1s/step	- accuracy: 0.81	
10/149	3:15	1s/step	- accuracy: 0.80	
11/149	3:13	1s/step	- accuracy: 0.81	
12/149	3:10	1s/step	- accuracy: 0.82	
13/149	3:08	1s/step	- accuracy: 0.82	
14/149	3:08	1s/step	- accuracy: 0.82	
15/149	3:07	1s/step	- accuracy: 0.82	

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py
 - requirements_classification.txt

brain_tumor_detection.py

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, \exclude, \escape)

Terminal

	python	+	-	...
96/149	1:03	1s/step	- accuracy: 0.82	
97/149	1:02	1s/step	- accuracy: 0.82	
98/149	1:01	1s/step	- accuracy: 0.82	
99/149	1:00	1s/step	- accuracy: 0.82	
100/149	59s	1s/step	- accuracy: 0.822	
101/149	58s	1s/step	- accuracy: 0.821	

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py
 - requirements_classification.txt

brain_tumor_detection.py

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, \exclude, \escape)

Terminal

	python	+	-	...
96/149	1:03	1s/step	- accuracy: 0.82	
97/149	1:02	1s/step	- accuracy: 0.82	
98/149	1:01	1s/step	- accuracy: 0.82	
99/149	1:00	1s/step	- accuracy: 0.82	
100/149	59s	1s/step	- accuracy: 0.822	
101/149	58s	1s/step	- accuracy: 0.821	

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal

Progress	Time	Step	Accuracy
33/149	2:15	1s/step	accuracy: 0.84
34/149	2:14	1s/step	accuracy: 0.84
35/149	2:13	1s/step	accuracy: 0.84
36/149	2:12	1s/step	accuracy: 0.84
37/149	2:10	1s/step	accuracy: 0.84
38/149	2:09	1s/step	accuracy: 0.84
39/149	2:08	1s/step	accuracy: 0.84

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal

Progress	Time	Step	Accuracy
68/149	1:34	1s/step	accuracy: 0.81
69/149	1:32	1s/step	accuracy: 0.81
70/149	1:31	1s/step	accuracy: 0.81
71/149	1:30	1s/step	accuracy: 0.81
72/149	1:29	1s/step	accuracy: 0.81
73/149	1:28	1s/step	accuracy: 0.81
74/149	1:27	1s/step	accuracy: 0.81

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal

Progress	Time	Step	Accuracy
38/149	2:37	1s/step	accuracy: 0.69
39/149	2:36	1s/step	accuracy: 0.69
40/149	2:35	1s/step	accuracy: 0.70
41/149	2:33	1s/step	accuracy: 0.70
42/149	2:32	1s/step	accuracy: 0.70
43/149	2:31	1s/step	accuracy: 0.70

The screenshot shows the VS Code interface with the file explorer on the left displaying a project structure: `brain-mri` (containing `data`, `tf_brain`, and `venv`), `brain_tumor_detection.py`, and `requirements_classification.txt`. The main editor displays the code for `brain_tumor_detection.py`, with lines 1235 to 1243 visible. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with a `python` session running. The terminal output shows progress bars and accuracy metrics for steps 38/149 through 43/149, with accuracy values ranging from 0.69 to 0.78.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Progress	Time	Accuracy
38/149	2:37 1s/step	accuracy: 0.69
39/149	2:36 1s/step	accuracy: 0.69
40/149	2:35 1s/step	accuracy: 0.70
41/149	2:33 1s/step	accuracy: 0.70
42/149	2:32 1s/step	accuracy: 0.70
43/149	2:31 1s/step	accuracy: 0.70

The screenshot shows the VS Code interface with the file explorer on the left displaying a project structure: `brain-mri` (containing `data`, `tf_brain`, and `venv`), `brain_tumor_detection.py`, and `requirements_classification.txt`. The main editor displays the code for `brain_tumor_detection.py`, with lines 1235 to 1243 visible. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with a `python` session running. The terminal output shows progress bars and accuracy metrics for steps 22/149 through 25/149, with accuracy values ranging from 0.30 to 0.32.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Progress	Time	Accuracy
22/149	3:25 2s/step	accuracy: 0.30
23/149	3:24 2s/step	accuracy: 0.31
24/149	3:23 2s/step	accuracy: 0.32
25/149	3:22 2s/step	accuracy: 0.32

The screenshot shows the VS Code interface with the file explorer on the left displaying a project structure: `brain-mri` (containing `data`, `tf_brain`, and `venv`), `brain_tumor_detection.py`, and `requirements_classification.txt`. The main editor displays the code for `brain_tumor_detection.py`, with lines 1235 to 1243 visible. The code includes a `main()` function call. The bottom panel shows the `Terminal` tab with a `python` session running. The terminal output shows progress bars and accuracy metrics for steps 148/149 and 149/149, with accuracy values of 0.6775 and 0.6767 respectively, and a loss of 0.8519. It also shows the start of Epoch 2.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Progress	Time	Accuracy	Loss
148/149	1s 1s/step	accuracy: 0.6775	
149/149	0s 1s/step	accuracy: 0.6767	
- loss: 0.8519			
Epoch 2: val_accuracy improved from 0.70119 to 0.79167, saving model to E:\brain-mri\venv\dist\50\brain_tumor_detection.py			

VS Code interface showing the execution of `brain_tumor_detection.py`. The Explorer sidebar shows the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor shows the code for `brain_tumor_detection.py` at line 1243, with the `main()` function being executed. The bottom panel shows the `Terminal` tab with the command `python ".\brain_tumor_detection.py"` and the output showing progress up to 28/149 steps with an accuracy of 0.72.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Step	Time	Step Time	Accuracy
25/149	2:21	1s/step	0.72
26/149	2:21	1s/step	0.73
27/149	2:20	1s/step	0.72
28/149	2:20	1s/step	0.72

VS Code interface showing the execution of `brain_tumor_detection.py`. The Explorer sidebar shows the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor shows the code for `brain_tumor_detection.py` at line 1243, with the `main()` function being executed. The bottom panel shows the `Terminal` tab with the command `python ".\brain_tumor_detection.py"` and the output showing progress up to 6/149 steps with an accuracy of 0.83.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Step	Time	Step Time	Accuracy
1/149	4:26	2s/step	0.78
2/149	3:42	2s/step	0.87
3/149	3:41	2s/step	0.87
4/149	3:40	2s/step	0.85
5/149	3:39	2s/step	0.85
6/149	3:37	2s/step	0.83

VS Code interface showing the execution of `brain_tumor_detection.py`. The Explorer sidebar shows the project structure: `brain-mri` (containing `data`, `tf_brain`, `venv`, `brain_tumor_detection.py`, and `requirements_classification.txt`). The main editor shows the code for `brain_tumor_detection.py` at line 1243, with the `main()` function being executed. The bottom panel shows the `Terminal` tab with the command `python ".\brain_tumor_detection.py"` and the output showing progress up to 64/149 steps with an accuracy of 0.80.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

Step	Time	Step Time	Accuracy
59/149	1:59	1s/step	0.80
60/149	1:57	1s/step	0.80
61/149	1:55	1s/step	0.80
62/149	1:54	1s/step	0.80
63/149	1:52	1s/step	0.80
64/149	1:51	1s/step	0.80

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py 6
 - requirements_classification.txt

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, lexclude, \escape)

Terminal

	python	+			
59/149			1:59	1s/step	- accuracy: 0.80
60/149			1:57	1s/step	- accuracy: 0.80
61/149			1:55	1s/step	- accuracy: 0.80
62/149			1:54	1s/step	- accuracy: 0.80
63/149			1:52	1s/step	- accuracy: 0.80
64/149			1:51	1s/step	- accuracy: 0.80

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py 6
 - requirements_classification.txt

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, lexclude, \escape)

Terminal

	python	+			
70/149			1:30	1s/step	- accuracy: 0.80
71/149			1:29	1s/step	- accuracy: 0.80
72/149			1:28	1s/step	- accuracy: 0.81
73/149			1:27	1s/step	- accuracy: 0.80
74/149			1:26	1s/step	- accuracy: 0.80
75/149			1:25	1s/step	- accuracy: 0.80
76/149			1:24	1s/step	- accuracy: 0.80

Explorer

- brain-mri
 - data
 - tf_brain
 - venv
 - brain_tumor_detection.py 6
 - requirements_classification.txt

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Problems 51 Output Terminal Ports Spell Checker 45

Debug Console

Filter (e.g. text, lexclude, \escape)

Terminal

	python	+			
59/149			2:05	1s/step	- accuracy: 0.82
60/149			2:04	1s/step	- accuracy: 0.82
61/149			2:02	1s/step	- accuracy: 0.82
62/149			2:00	1s/step	- accuracy: 0.82
63/149			1:59	1s/step	- accuracy: 0.82
64/149			1:57	1s/step	- accuracy: 0.82

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Step	Time	Loss	Accuracy
22/149	2:08	1s/step	accuracy: 0.76
23/149	2:07	1s/step	accuracy: 0.75
24/149	2:06	1s/step	accuracy: 0.76
25/149	2:05	1s/step	accuracy: 0.76
26/149	2:04	1s/step	accuracy: 0.76
27/149	2:03	1s/step	accuracy: 0.76
28/149	2:02	1s/step	accuracy: 0.76

Step	Time	Loss	Accuracy
9/149	3:16	1s/step	accuracy: 0.81
10/149	3:15	1s/step	accuracy: 0.80
11/149	3:13	1s/step	accuracy: 0.81
12/149	3:10	1s/step	accuracy: 0.82
13/149	3:08	1s/step	accuracy: 0.82
14/149	3:08	1s/step	accuracy: 0.82
15/149	3:07	1s/step	accuracy: 0.82

Step	Time	Loss	Accuracy
68/149	1:41	1s/step	accuracy: 0.81
69/149	1:39	1s/step	accuracy: 0.81
70/149	1:38	1s/step	accuracy: 0.81
71/149	1:36	1s/step	accuracy: 0.81
72/149	1:35	1s/step	accuracy: 0.81
73/149	1:33	1s/step	accuracy: 0.81
74/149	1:32	1s/step	accuracy: 0.81

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, tf_brain, venv, brain_tumor_detection.py, and requirements_classification.txt. The main editor shows the code for brain_tumor_detection.py, with lines 1235 to 1243 visible. The terminal at the bottom shows the output of the program, including accuracy and loss values for epochs 146 through 149.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

```
146/149 ██████████ 3s 1s/step - accuracy: 0.8139
147/149 ██████████ 2s 1s/step - accuracy: 0.8141
148/149 ██████████ 1s 1s/step - accuracy: 0.8132
149/149 ██████████ 0s 1s/step - accuracy: 0.8141
- loss: 0.4983
Epoch 8: val_accuracy improved from 0.85595 to 0.86071, saving model to E:\my-data\Openpy-fish-stuff\projects-for-ppa
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, tf_brain, venv, brain_tumor_detection.py, and requirements_classification.txt. The main editor shows the code for brain_tumor_detection.py, with lines 1235 to 1243 visible. The terminal at the bottom shows the output of the program, including accuracy and loss values for epochs 96 through 102.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

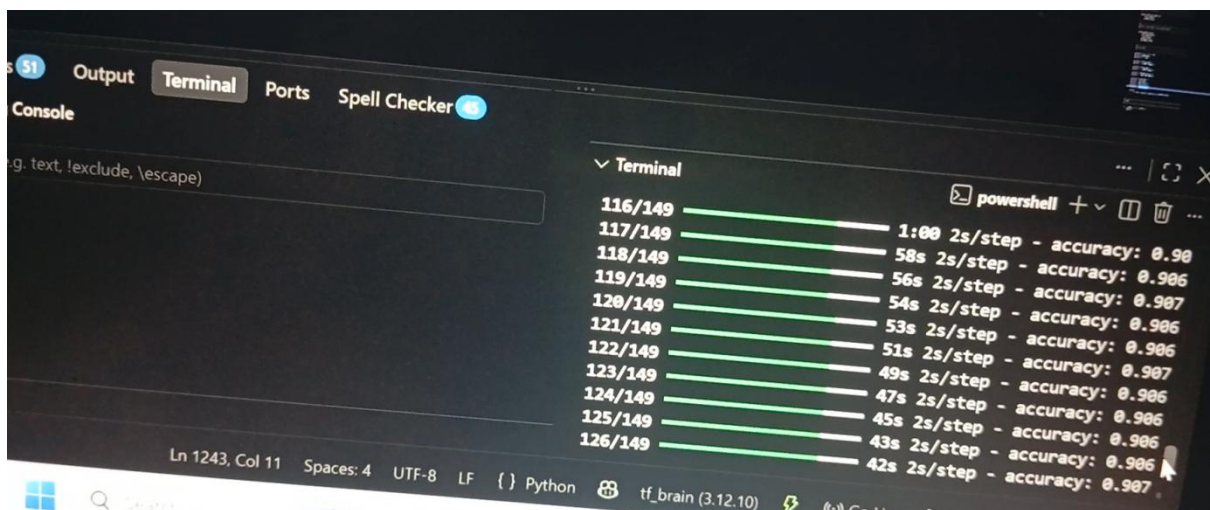
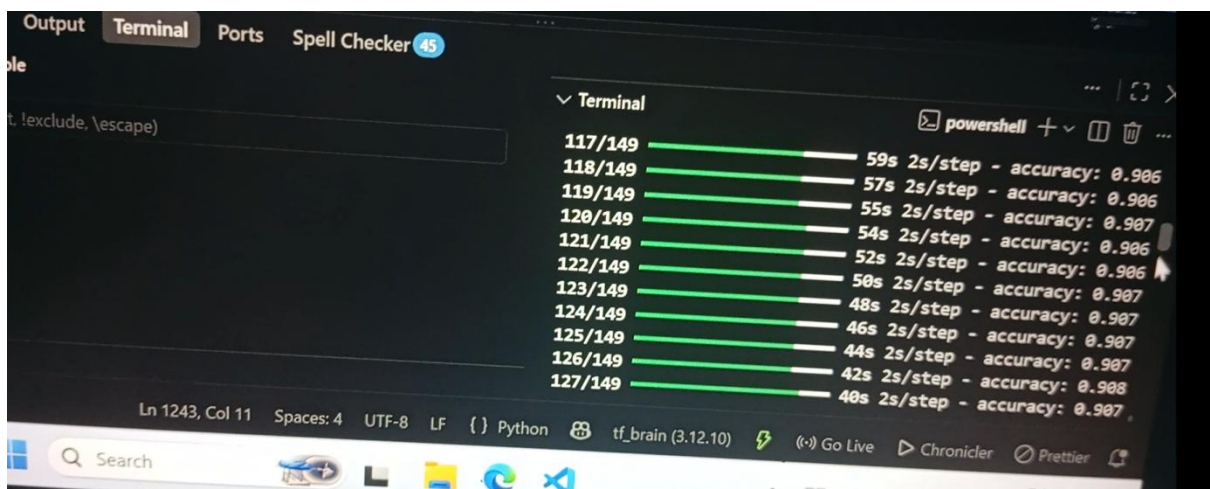
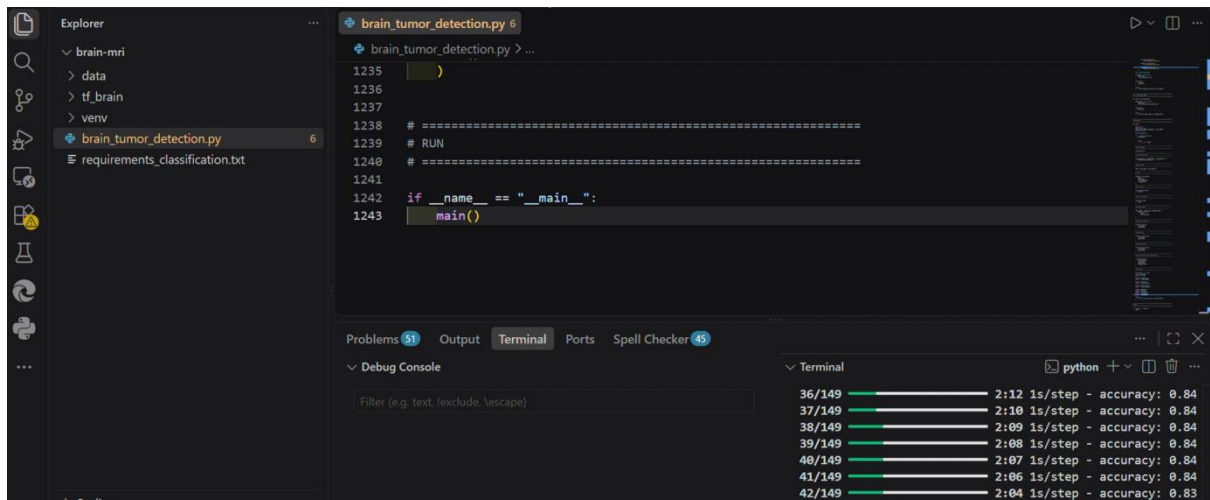
```
96/149 ██████████ 1:03 1s/step - accuracy: 0.82
97/149 ██████████ 1:02 1s/step - accuracy: 0.82
98/149 ██████████ 1:01 1s/step - accuracy: 0.82
99/149 ██████████ 1:00 1s/step - accuracy: 0.82
100/149 ██████████ 59s 1s/step - accuracy: 0.822
101/149 ██████████ 58s 1s/step - accuracy: 0.821
102/149 ██████████ 57s 1s/step - accuracy: 0.821
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure: brain-mri, data, tf_brain, venv, brain_tumor_detection.py, and requirements_classification.txt. The main editor shows the code for brain_tumor_detection.py, with lines 1235 to 1243 visible. The terminal at the bottom shows the output of the program, including accuracy and loss values for epochs 5 through 11.

```
1235 )
1236
1237
1238 # =====
1239 # RUN
1240 # =====
1241
1242 if __name__ == "__main__":
1243     main()
```

Terminal Output:

```
5/149 ██████████ 3:20 1s/step - accuracy: 0.83
6/149 ██████████ 3:09 1s/step - accuracy: 0.85
7/149 ██████████ 3:04 1s/step - accuracy: 0.85
8/149 ██████████ 3:00 1s/step - accuracy: 0.85
9/149 ██████████ 2:54 1s/step - accuracy: 0.85
10/149 ██████████ 2:50 1s/step - accuracy: 0.84
11/149 ██████████ 2:47 1s/step - accuracy: 0.84
```



Output Terminal Ports Spell Checker

onsole

text, !exclude, \escape)

Terminal

146/149 5s 2s/step - accuracy: 0.9099

147/149 3s 2s/step - accuracy: 0.9101

148/149 1s 2s/step - accuracy: 0.9105

149/149 0s 2s/step - accuracy: 0.9105

- loss: 0.2460

Epoch 39: val_accuracy improved from 0.91071 to 0.91310, saving model to E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output\best_brain_tumor_model.keras

Epoch 39: finished saving model to E:\my-data\Opencv-job-s

Ln 1243, Col 11 Spaces: 4 UTF-8 LF {} Python

Terminal Ports Spell Checker

Terminal

132/149 31s 2s/step - accuracy: 0.907

133/149 29s 2s/step - accuracy: 0.907

134/149 27s 2s/step - accuracy: 0.907

135/149 25s 2s/step - accuracy: 0.907

136/149 23s 2s/step - accuracy: 0.907

137/149 21s 2s/step - accuracy: 0.907

138/149 20s 2s/step - accuracy: 0.907

139/149 18s 2s/step - accuracy: 0.907

140/149 16s 2s/step - accuracy: 0.905

141/149 14s 2s/step - accuracy: 0.905

142/149 12s 2s/step - accuracy: 0.905

Ln 1243, Col 11 Spaces: 4 UTF-8 LF {} Python tf_brain (3.12.10) Go Live Chronicer Prettier 7:10 PM

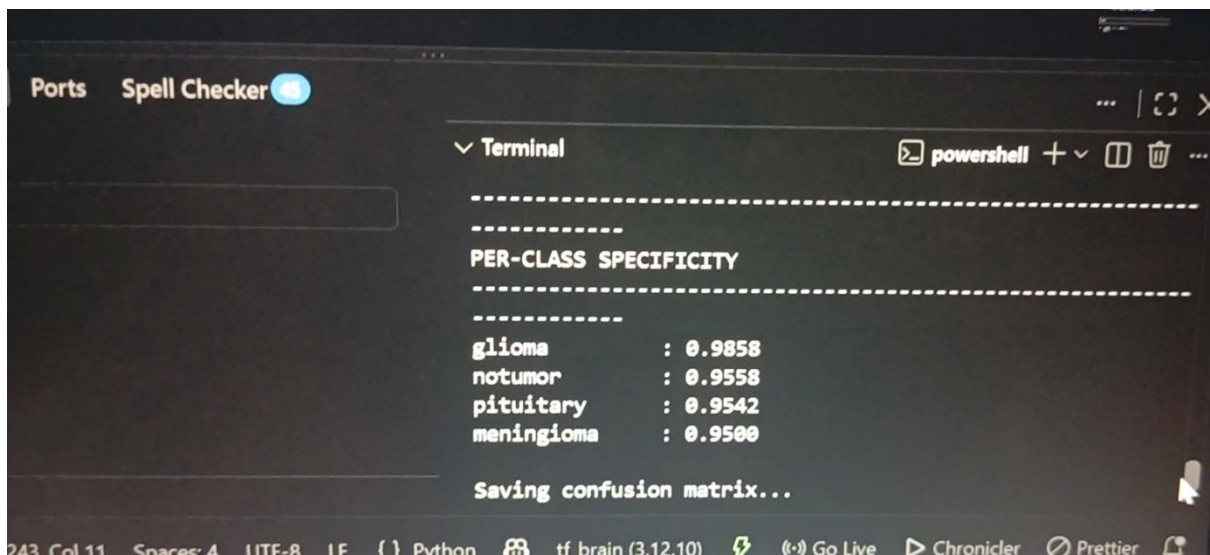
Output Terminal Ports Spell Checker

Terminal

precision recall f1-score support

Glioma	0.9446	0.7250	0.8204	400
No Tumor	0.8822	0.9925	0.9341	400
Pituitary	0.8780	0.9900	0.9307	400
Meningioma	0.8469	0.8300	0.8384	400
accuracy			0.8844	1600
macro avg	0.8880	0.8844	0.8809	1600
weighted avg	0.8880	0.8844	0.8809	1600

Ln 1243, Col 11 Spaces: 4 UTF-8 LF {} Python tf_brain (3.12.10) Go Live Chronicer Prettier

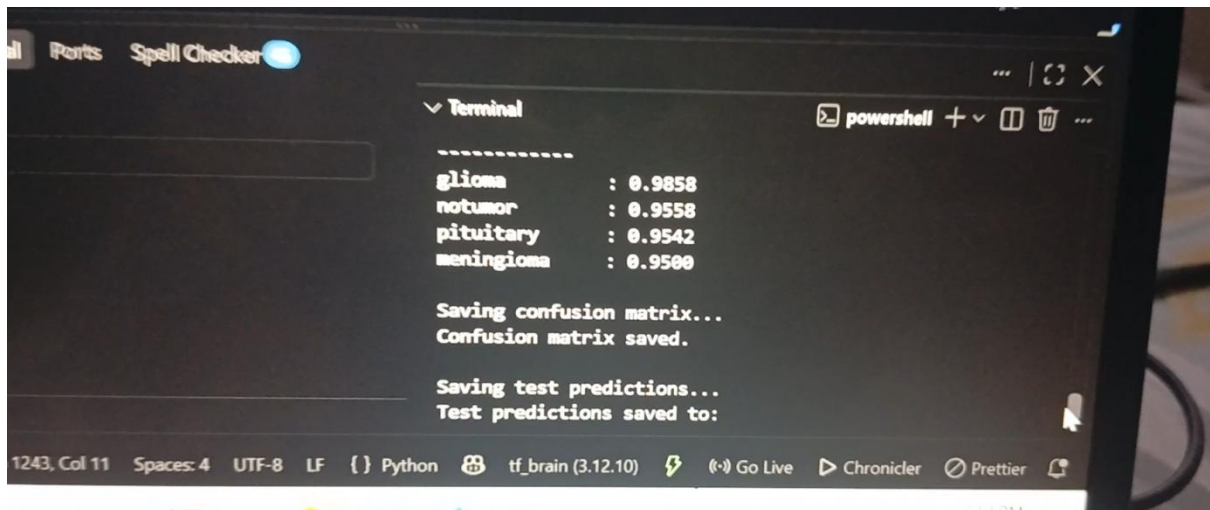


Ports Spell Checker

Terminal powershell

```
-----  
PER-CLASS SPECIFICITY  
-----  
glioma      : 0.9858  
notumor     : 0.9558  
pituitary   : 0.9542  
meningioma  : 0.9500  
  
Saving confusion matrix...
```

243, Col 11 Spaces: 4 UTF-8 LF { } Python tf_brain (3.12.10) Go Live Chronicer Prettier

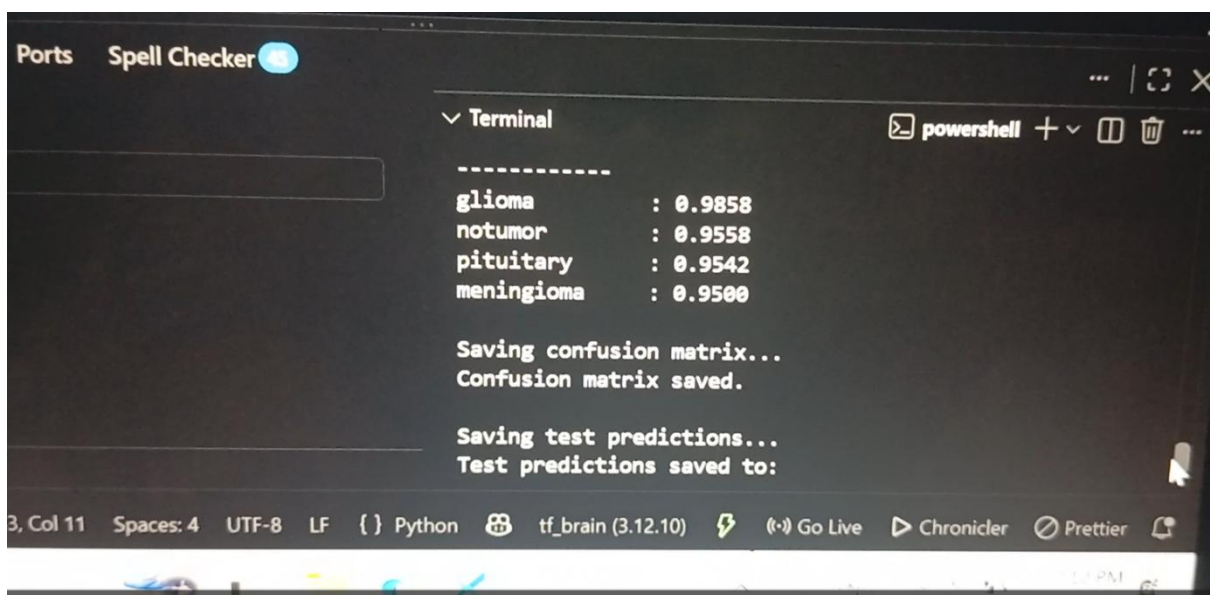


Ports Spell Checker

Terminal powershell

```
-----  
glioma      : 0.9858  
notumor     : 0.9558  
pituitary   : 0.9542  
meningioma  : 0.9500  
  
Saving confusion matrix...  
Confusion matrix saved.  
  
Saving test predictions...  
Test predictions saved to:
```

1243, Col 11 Spaces: 4 UTF-8 LF { } Python tf_brain (3.12.10) Go Live Chronicer Prettier



Ports Spell Checker

Terminal powershell

```
-----  
glioma      : 0.9858  
notumor     : 0.9558  
pituitary   : 0.9542  
meningioma  : 0.9500  
  
Saving confusion matrix...  
Confusion matrix saved.  
  
Saving test predictions...  
Test predictions saved to:
```

3, Col 11 Spaces: 4 UTF-8 LF { } Python tf_brain (3.12.10) Go Live Chronicer Prettier

al Ports Spell Checker

Terminal powershell

Classes:
Glioma
No Tumor
Pituitary
Meningioma

All results saved in:
E:\my-data\Opencv-job-stuff\projects-for-programming-learning\portfolio-projects\brain-mri\data\output
○ (tf_brain) PS D:\brain-mri>

1243, Col 11 Spaces: 4 UTF-8 LF { } Python tf_brain (3.12.10) Go Live Chronieler Prettier

